

ARGOS

Sprint Manual — Enzo Ordoñez Flores

Líder Técnico — SOAR · HITL · Notificaciones

Curso: Tópicos Avanzados de Ciberseguridad · Universidad de Lima ·
2026-1

Entrega final: 13 de junio de 2026

Versión 2.0 — manual organizado por orden de implementación (Fase 1 →
4)

Parte I — Introducción común al proyecto

ARGOS — Introducción al proyecto y arquitectura

Documento común a todos los manuales de integrante. Contiene contexto que cada miembro del equipo (P1/P2/P3/P4) necesita antes de leer su manual individual.

Field	Value
Type	Manual de introducción común para los 4 integrantes
Status	Active
Última actualización	2026-05-24
Pre-requisito	Ninguno. Léelo de cero.

1. Qué es ARGOS en una página

ARGOS es la **Adaptive Response Guard with Orchestrated Surveillance** — una plataforma multi-vector de detección y respuesta a amenazas que replica la arquitectura de productos comerciales high-end EDR/XDR (Microsoft Defender XDR, CrowdStrike Falcon, Palo Alto Cortex XDR) usando exclusivamente componentes open source más una API LLM de bajo costo para la capa de triage.

El proyecto es para el curso **Tópicos Avanzados de Ciberseguridad** en la Universidad de Lima · 2026-1. La entrega final es el **13 de junio de 2026** y consta de tres deliverables obligatorios: informe técnico (~30% del peso), demo en vivo (~40%) y presentación (~20%) más los seguimientos en semanas 5/7/9 (~10%).

El activo defendido es una base de datos **PostgreSQL 15** corriendo sobre una Linux VM en el lab. Tiene esquema `argos_demo_prod` con tablas que simulan datos de RRHH y finanzas (employees, payroll, customers, invoices, payments) con datos sintéticos. El host está tagged en Wazuh como `criticality=production-critical`, lo que dispara la regla de dos personas (two-person rule) para cualquier acción de containment sobre él.

El alcance multi-vector (post ADR-0008) cubre tres categorías de ataques distintas:

- **Ransomware:** cifrado de archivos via técnicas T1486/T1490/T1083/T1562. Es el énfasis primario.

- **Network Denial of Service:** ataques de red contra el puerto del servicio (T1498/T1499) con `hping3` o `slowhttptest`.
- **Application Abuse:** SQL injection contra una app web delante de la DB (T1190) usando `sqlmap`, y false positives de actividad legítima de usuario (T1078 Valid Accounts) — un SELECT masivo a las 3 AM se parece a exfiltración pero puede ser un reporte mensual legítimo.

Por qué este alcance y no más: ARGOS no busca cobertura exhaustiva multi-vector. Busca cobertura mínima representativa que demuestre que la arquitectura de 4 capas + SOAR + HITL es genuinamente adaptativa. Cubrir 3 vectores con profundidad real es más defendible que cubrir 10 superficialmente.

2. Arquitectura de 4 capas defensivas

ARGOS sigue el patrón industrial estándar de **defense-in-depth**: cuatro capas independientes y paralelas, cada una con trade-offs distintos, fallan independientemente. Si una capa cae o es evadida, las otras tres mantienen cobertura. No hay un solo punto de falla en la detección.



Capa 1 — Rule-Based Detection (Sigma + Wazuh)

Dueño: P3 (Angeles). Reglas YAML escritas en formato Sigma, convertidas a Wazuh con `sigma-cli`, mapeadas explícitamente a técnicas MITRE ATT&CK. Detectan patrones conocidos.

Sub-categorías por dominio (post ADR-0008):

- `detection/sigma-rules/ransomware/` — vssadmin, T1486 cifrado, T1083 enumeración, T1562 disable defender, T1021 lateral, T1071 C2
- `detection/sigma-rules/network/` — rate-based rules para T1498/T1499 (DDoS, slow-rate)
- `detection/sigma-rules/database/` — query pattern anomalies para UC-07
- `detection/sigma-rules/webapp/` — T1190 SQL injection signatures

Trade-off honesto: alta precisión, recall limitado contra variantes nuevas. Por eso existe Capa 2.

Capa 2 — ML Anomaly Detection

Dueño: P2 (Sebastian). Modelos no supervisados entrenados sobre baseline benigno. Detectan desviaciones que las reglas no captan.

Modelos especializados por dominio (post ADR-0008):

- `ml/models/ransomware_ensemble.pkl` — Isolation Forest + One-Class SVM. Features ventana 60s: file_write_rate, avg_entropy (Shannon), extension_modification_ratio, crypto_api_calls, new_outbound_connections, cpu_burst_score, io_burst_score.
- `ml/models/network_traffic_anomaly.pkl` — para UC-06 DDoS. Features: connections/sec, packet rate, source IP entropy, packet size variance.
- `ml/models/query_pattern_anomaly.pkl` — para UC-07 SELECT masivo. Features: rows_returned, query_duration_ms, hour_of_day, user_id, query_template_hash.

Ensemble ransomware: $\text{ensemble_score} = 0.6 \times \text{isolation_forest_score} + 0.4 \times \text{one_class_svm_score}$. Los demás modelos retornan score único.

Trade-off honesto: mayor recall contra variantes nuevas, mayor false positive rate. Requiere baseline limpio.

Capa 3 — Canary Deception

Dueño: P3 (Angeles). Archivos-cebo (`financials_Q4_2025.xlsx` , `passwords.txt` , `db_backup.sql`) en ubicaciones donde un usuario legítimo nunca tocaría. FIM (File Integrity Monitoring) con Sysmon whodata (Windows) o auditd (Linux) captura QUÉ proceso accedió.

Lógica: primera modificación / lectura / rename de un canary = alerta crítica con confianza máxima. Por diseño, FP rate ≈ 0 .

Esta capa es estrictamente ransomware-specific. No tiene sub-categorías por dominio. Documentado deliberadamente en ADR-0008: defender contra DDoS o SQL injection requiere otras primitivas, no canaries.

Capa 4 — LLM-Assisted Triage

Dueño: P1 (Enzo). FastAPI service que recibe el contexto completo de una alerta (process tree, network connections, file modifications) y produce análisis estructurado: técnica MITRE, severidad, runbook NIST aplicable, acción recomendada, IoCs a correlar.

Backend vendor-agnostic (per ADR-0001 v2):

- **Primary:** OpenAI GPT-4o-mini (US-based, soberanía de datos aceptable)
- **Fallback:** Llama 3.1 8B local vía Ollama (zero-egress, funciona sin internet)

Invariante crítico R-02: el LLM **NUNCA** está en el **path crítico de containment**. El SOAR decide desde Capas 1-3 solamente. El LLM enriquece la vista del analista; si halucina, miente, o cae, el sistema sigue funcionando.

Pipeline RAG: BM25 (léxico) + BGE-large embeddings (denso) + RRF (Reciprocal Rank Fusion). Hybrid retrieval estándar industrial. Sin cross-encoder reranker (descartado en ADR-0001 v2 por marginal gain vs costo).

3. SOAR Decision Engine y los 4 tiers de confianza

El **SOAR (Security Orchestration, Automation and Response)** es el cerebro que fusiona señales de las 4 capas y decide la acción. Lo dueño es **P1**. Implementa la lógica de tiers per ADR-0003.

Tier	Disparado por	Confianza	Acción
T0	Canary solo, OR las 3 capas corroboran	≥ 0.95	Aislamiento automático inmediato + snapshot
T1	Layer 1 + Layer 2 corroboran (sin canary)	0.80–0.95	Aislamiento automático inmediato + snapshot
T2	Capa sola con score alto	0.60–0.80	Throttle + snapshot ahora , aislamiento full pendiente de aprobación 3-min
T3	Corroboración baja	0.40–0.60	Solo notificación enriquecida con LLM

(*) **Los thresholds son preliminares** pendientes de calibración empírica per `OPEN_QUESTIONS_RESOLUTION.md` §Q5.

Tier T2 — el más interesante

Ransomware moderno cifra ~25,000 archivos/min. Un countdown de 3 minutos para aprobación humana sin protección parece estúpido. Por eso T2 NO es "pendiente": es "throttle + snapshot AHORA, full isolation pendiente de aprobación". El throttle (cpulimit/ionice) reduce la tasa de cifrado a ~100-500 files/min mientras los aprobadores ven el contexto LLM y deciden. Si el timeout de 3 min expira sin respuesta, el sistema auto-ejecuta (no espera más). Si el aprobador clickea Reject, el throttle se levanta y el caso queda registrado como false positive con razón documentada — esto es UC-07.

Split-brain (conflicto multi-aprobador)

Per ADR-0006: cuando hay 4 aprobadores y dan respuestas contradictorias (2 approve, 1 reject, 1 timeout), se aplica **conservative-wins policy** con ventana de consolidación de 60 segundos: cualquier "approve" gana sobre rechazos, excepto para acciones irreversibles que requieren **two-person rule** (dos approves explícitos, un reject cancela).

El **two-person rule** se activa cuando el host afectado tiene `criticality=production-critical` (nuestro PostgreSQL). Esto es UC-04 en el demo.

Canales de notificación (per ADR-0007 v2)

Tiempo	Canal	Propósito
t=0	Telegram bot con botones inline JWT	Primario — push agresivo a celulares de aprobadores
t=0	Discord webhook con @-mention de role	Visibilidad en server del equipo, presión social
t=60s	Twilio Voice DTMF (sin STT)	Escalación si nadie respondió. "Presione 1 para aprobar, 2 para rechazar"
post-decisión	Email	Resumen asíncrono, NUNCA en path crítico

4. Equipo y responsabilidades

	Integrante	Rol	Alcance principal
	Enzo Ordoñez Flores	P1 · Líder · LLM/ SOAR	<code>argos_contracts</code> (entregado), Capa 4 LLM Triage, motor SOAR + Tier Classifier, Approval API con JWT, notificaciones multi-canal, Consola de Aprobación Streamlit, simulador de ransomware, playbooks de containment, coordinación
	Sebastian Montenegro	P2 · Ingeniero ML	Capa 2 completa (Isolation Forest + One-Class SVM ensemble + modelos especializados), feature extraction, calibración thresholds, métricas P/R/F1, captura forense
	Angeles Castillo	P3 · Detección y Engaño	Capa 1 (Sigma rules ransomware + network + database + webapp), Capa 3 (canary FIM + whodata), validación con Atomic Red Team y Caldera, bonus: PRs upstream a SigmaHQ
	Diego Jara	P4 · Infraestructura	Vagrantfile + Wazuh/OpenSearch/Redis deployment, PostgreSQL con datos sintéticos, simuladores (ransomware + DDoS + SQL injection), UI Streamlit base, video demo

Regla operativa crítica: cada integrante debe poder defender su capa en exposición. P1 no escribe código de otros con Claude Code. Cada integrante puede usar Claude Code (o cualquier asistente IA) como **acelerador en SU propia parte**, siempre que entienda lo que produce y pueda defenderlo en viva (per CONTEXT.md §5 reinterpretado post ADR-0008).

5. Los 8 use cases del demo

El demo en vivo cubre 8 escenarios (per ADR-0008 multi-vector). Cada UC tiene narrativa específica que demuestra una propiedad distinta del sistema.

UC	Vector	Tier	Capas que firing	Qué demuestra
UC-01	Ransomware (LockBit-like)	T0	1+2+3	Full-stack end-to-end. Las 3 capas firing simultáneo. Auto-isolate + email post-facto.
UC-02	Canary deception	T0	3 sola	Detección ultra-temprana. Zero archivos reales cifrados.
UC-03	Variante novel + split-brain	T2	2 sola	Centerpiece HITL ransomware. ML atrapa lo que reglas no. 4 aprobadores. Conservative-wins live.
UC-04	PostgreSQL + two-person rule	T1	1+2	Compliance vocabulary. Four-eyes principle. Governance.
UC-05	Stealth attack (agent-kill)	T0	1+3+heartbeat	Resiliencia: agent disconnect = signal.
UC-06	DDoS volumetric	T0	1 (rate)+2 (network)	Cobertura multi-vector. Defiende contra ataques de red, no solo endpoint. T1498.
UC-07	SELECT masivo legítimo FP	T2	2 sola	Pieza clave del HITL. Humano cancela contención por FP reconocido. T1078.
UC-08	SQL injection	T1	1+2	OWASP Top 10 #1. T1190 Exploit Public-Facing Application.

Los dos son los UCs que más venden el HITL. UC-03 muestra split-brain con conservative-wins; UC-07 muestra cancelación de FP por humano. Ambos son lo que diferencia un EDR/XDR con HITL de un SIEM tradicional.

6. Ejemplo end-to-end de UC-01 paso a paso

Para que tengas un modelo mental concreto de cómo fluye una alerta a través del sistema, aquí está UC-01 (ransomware clásico LockBit-like) desglosado segundo a segundo.

Setup pre-ataque (T-5 min)

- Lab está corriendo: Wazuh manager + Windows VM + Linux VM con PostgreSQL.
- Los 4 aprobadores tienen Telegram abierto en sus celulares.
- La pantalla del proyector muestra: Streamlit Approval Console (vacía) + terminal de P4 listo para ejecutar el simulador.
- P1 narra el contexto: "Este es nuestro endpoint Windows típico de la organización. Tiene documentos en `C:\Users\Demo\Documents\`. El atacante acaba de obtener acceso inicial."

T=0 — Atacante ejecuta

P4 corre:

```
python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim --speed full
```

Esto inicia el simulador. Behavior chain:

1. T+1s: enumera archivos en `C:\Users\Demo\Documents\` (técnica T1083 File and Directory Discovery)
2. T+2s: invoca `vssadmin delete shadows /all /quiet` (técnica T1490 Inhibit System Recovery)
3. T+3s: comienza a cifrar archivos con AES-256, renombrando a `.locked` (técnica T1486 Data Encrypted for Impact)
4. T+4s: toca el canary file `financials_Q4_2025.xlsx`
5. T+5s: deja la ransom note `README_RESTORE_FILES.txt` en el desktop

T+1 a T+5 segundos — Capas detectan en paralelo

Capa 1 (Sigma + Wazuh) detecta:

- T+2s: regla `T1490_vssadmin_delete_shadows` dispara cuando ve el comando `vssadmin`.
- T+3s: regla `T1486_mass_file_encryption_extension` dispara al ver renames masivos a `.locked`.
- Wazuh manager normaliza estas alertas y las publica al stream Redis `wazuh:alerts`.

Capa 2 (ML) detecta:

- ML consumer (Python proceso corriendo en bg) lee el stream Redis.
- Cada 60s genera features para los procesos activos. La ventana T+1 a T+60 captura el simulator process.
- Features observadas: `file_write_rate=347` archivos/min, `avg_entropy=7.8` (alta), `extension_modification_ratio=0.95`, `crypto_api_calls=42`, `cpu_burst_score=0.9`.
- Isolation Forest score: 0.94. One-Class SVM score: 0.91. Ensemble: 0.93.
- Publica `MLScore` con score 0.93 al stream Redis `ml:scores`.

Capa 3 (Canary FIM) detecta:

- T+4s: el simulador toca `financials_Q4_2025.xlsx`.
- Sysmon whodata captura el evento con PID del simulador y command-line completo.
- Wazuh dispara regla custom `canary_file_accessed` con severity 12 (crítica).
- Score implícito: 1.0 (canary fire = certeza máxima).

T+5 segundos — SOAR Decision Engine fusiona

El **SOAR orchestrator** (proceso de P1 corriendo el FastAPI Approval API en port 8003) se entera de las 3 alertas dentro del mismo incident window de 5 segundos.

Tier Classifier (`soar/decision_engine/tier_classifier.py`) aplica reglas:

- `canary_fired=True` → tier T0 (canary alone wins to T0).
- Pero también L1 + L2 corroboran con high scores → tier T0 confirmed.
- **Decisión:** `Tier.T0` con confidence 0.95+.

State machine (`soar/decision_engine/state_machine.py`):

1. Crea Incident con `incident_id=INC-2026-05-26-001`, state `RECEIVED`.
2. Persiste en Redis con key `incident:INC-2026-05-26-001`.
3. Llama al LLM Triage en paralelo (Capa 4 enrichment-only, NO bloquea decisión).
4. Transición de estado: `RECEIVED` → `PENDING_EXECUTION`.

Capa 4 LLM Triage (en paralelo, T+5 a T+8s):

- FastAPI `/triage` endpoint recibe el AlertContext con las 3 alertas.
- Llama a OpenAI GPT-4o-mini con prompt enriquecido.
- Devuelve TriageResponse: `tecnica_mitre=T1486`, `confianza=0.94`, `severidad=critical`, `runbook_aplicable="NIST 800-61 §3.4 Containment"`, `accion_recomendada="Isolate host immediately and capture memory snapshot before remediation"`.
- El Incident se actualiza en Redis con el `llm_analysis` campo.

T+8 segundos — Playbook automático

Como es T0 sin override de criticality (es una Windows VM standard, no production-critical), no requiere approval. El SOAR ejecuta el playbook directamente:

1. **Host isolation** (`soar/playbooks/host_isolation.py`): conecta vía PowerShell al Windows VM y crea regla firewall: `New-NetFirewallRule -DisplayName "ARGOS-Isolation" -Direction Outbound -Action Block`.
2. **Process kill**: identifica el PID del simulador desde el whodata FIM, ejecuta `Stop-Process -Force -Id <PID>`.
3. **Disk snapshot**: invoca Volume Shadow Copy `vssadmin create shadow /for=C:` para preservar evidencia forense.

4. Transición de estado: `PENDING_EXECUTION` → `EXECUTED` .

T+10 segundos — Notificación post-facto

El SOAR envía notificación multi-canal (per ADR-0007 v2):

- **Telegram** a los 4 chat_ids configurados: mensaje con incident_id, técnica, análisis LLM, y botón inline "Revertir" firmado con JWT.
- **Discord** webhook al server del equipo con embed coloreado por tier (T0 = rojo) y `@argos-approvers` mention.
- **Email post-facto** a `APPROVER_EMAILS` (los 4 emails del equipo) con resumen + link al audit log en OpenSearch.

T+10 a T+30 segundos — Streamlit Console muestra

La **Streamlit Approval Console** (proceso de P1 corriendo en port 8501, proyectado en pantalla) refresca cada 2 segundos vía `streamlit-autorefresh` . Muestra:

- **Panel izquierdo (Incident Card):** badge T0 en rojo, incident_id, host afectado, técnica MITRE T1486, análisis LLM compacto.
- **Panel central (Decision Matrix):** vacío porque es T0 auto-execute, no requirió aprobación.
- **Panel derecho (System Logic):** estado actual `EXECUTED` , decisión `EXECUTE_ISOLATION` , política aplicada `auto-execute` , justificación "T0 confirmed: canary + L1 + L2 corroborate".
- **Panel inferior (Action Timeline):** línea de tiempo horizontal mostrando: alert created → tier classified → playbook executed → notifications sent.

T+30 segundos — Análisis forense visible

P1 narra: "El sistema aisló el host en menos de 10 segundos. 31 archivos cifrados de 500. El canary salvó los restantes. Los 4 aprobadores tienen el incidente en su Telegram con la opción de Revertir si fuera falso. Audit log completo en OpenSearch con cada decisión justificada."

Métricas que se muestran en pantalla

- **TTD (Time to Detect):** 4.2 segundos desde T=0 hasta primera alerta válida.
- **TTI (Time to Isolate):** 8.7 segundos desde T=0 hasta playbook completado.
- **Archivos cifrados antes de containment:** 31 de 500 (94% preservados).
- **MITRE coverage:** T1083 + T1490 + T1486 cubiertos por L1, ratificados por L2 entropy spike, confirmados por canary L3.

Lo que NO se ve pero pasó

- LLM Triage tardó ~3 segundos en responder. NUNCA bloqueó la decisión de containment.

- El throttle preventivo NO se aplicó porque fue T0 (auto-execute directo). En T2 sí se habría aplicado mientras corre el countdown.
 - El audit log en OpenSearch capturó: triggering layers, scores individuales, fusion logic, LLM analysis, playbook commands executed, notification channels disparados, timestamps de cada paso.
-

7. Glosario MITRE ATT&CK

Las técnicas relevantes al alcance del proyecto, ordenadas por categoría (tactic).

Initial Access

- **T1078 — Valid Accounts:** atacante usa credenciales legítimas. Relevante para UC-07 (escenario FP: usuario legítimo, NO atacante).
- **T1190 — Exploit Public-Facing Application:** explotación de vulnerabilidad en aplicación web expuesta. T1190.001 sub-técnica = SQL Injection. Relevante para UC-08.

Defense Evasion

- **T1562 — Impair Defenses:** atacante deshabilita herramientas de seguridad. T1562.001 sub-técnica = Disable Defender. Relevante para UC-05 (agent-kill attempt).
- **T1070 — Indicator Removal:** atacante borra rastros. T1070.001 = Clear Windows Event Logs, T1070.004 = File Deletion.

Discovery

- **T1083 — File and Directory Discovery:** ransomware enumera archivos antes de cifrar. Relevante para UC-01.

Lateral Movement

- **T1021 — Remote Services:** SMB/RDP/SSH usados por atacante para moverse. T1021.001 = RDP, T1021.002 = SMB, T1021.004 = SSH.

Command and Control

- **T1071 — Application Layer Protocol:** beacon a C2 server vía HTTP/HTTPS. T1071.001 sub-técnica = Web Protocols.

Impact (la más rica para ARGOS)

- **T1486 — Data Encrypted for Impact:** ransomware cifrando archivos. Núcleo de UC-01, UC-02, UC-03.
 - **T1490 — Inhibit System Recovery:** borrar Volume Shadow Copies, snapshots btrfs, etc. Precursor a T1486.
 - **T1498 — Network Denial of Service:** ataques de red contra disponibilidad. T1498.001 = Direct Network Flood (UC-06), T1498.002 = Reflection Amplification.
 - **T1499 — Endpoint Denial of Service:** saturación de recursos del endpoint. T1499.002 = Service Exhaustion, T1499.004 = Application Exploitation.
-

8. Stack tecnológico completo

Categoría	Componente	Función	Owner principal
SIEM	Wazuh 4.7	Manager + agentes	P4
SIEM	OpenSearch	Storage de alertas y audit log	P4
SIEM	Sigma + sigma-cli	Reglas de detección	P3
Telemetría	Sysmon (Windows)	Eventos detallados de proceso/ archivo/red	P4
Telemetría	auditd (Linux)	Eventos detallados de syscall	P4
Telemetría	pgAudit (PostgreSQL)	Audit log de queries	P4
Telemetría	Wazuh FIM whodata	Captura qué proceso tocó archivo (canary)	P3
Simulación	Atomic Red Team	TTPs MITRE 1-a-1	P3
Simulación	Caldera	Cadenas de ataque multi-step	P3
Simulación	Custom ransomware simulator (Python)	UC-01, UC-02, UC-03 reproducibles	P1
Simulación	hping3 + slowhttptest	UC-06 DDoS	P4
Simulación	sqlmap	UC-08 SQL injection	P4
ML	scikit-learn 1.4+	Isolation Forest + One-Class SVM	P2
ML	scipy	Shannon entropy	P2
ML	pandas + numpy	Data wrangling	P2
ML	joblib	Model serialization	P2
Backend	FastAPI	LLM Triage API + Approval API	P1
Backend	Redis 7+	Alert bus + incident state machine	P4 (deploy) / P1 (usage)
Backend	APScheduler	Timeouts + consolidation windows	P1
Backend	PyJWT	JWT signing para approval tokens	P1
Backend	Pydantic v2	argos_contracts (cross-team interfaces)	P1 (shipped)
LLM	OpenAI GPT-4o-mini	Primary backend (cloud, US-based)	P1
LLM	Llama 3.1 8B vía Ollama	Fallback (local, zero-egress)	P1
LLM	BGE-large embeddings	Retrieval en RAG	P1
Notifications	python-telegram-bot	Telegram bot con inline buttons	P1
Notifications	discord-webhook	Discord notifications con role mention	P1
Notifications	twilio	Voice DTMF escalación	P1

Categoría	Componente	Función	Owner principal
UI	Streamlit 1.30+	Analyst Console + Approval Console	P4 (base) / P1 (Approval)
UI	OpenSearch Dashboards	MITRE heatmap, TTD histogram, layer perf	P4
Infra	Vagrant 2.4+	Provisioning de 3 VMs	P4
Infra	VirtualBox 7.x	Hypervisor para lab local	P4
Testing	pytest + respx + fakeredis	Tests cross-module	Todos
DB	PostgreSQL 15	Activo defendido	P4 (deploy + datos sintéticos)

9. Convenciones del repo

Git workflow

- **Branch principal:** `main`, protegida en GitHub.
- **Feature branches:** `feature/<persona>/<descripcion-corta>` — ejemplo: `feature/p2/isolation-forest-baseline`.
- **Commits convencionales:** `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.
- **PRs requieren:** CI verde + 1 review. Pairing: P1 ↔ P2, P3 ↔ P4.
- **Push diario obligatorio** antes de las 22:00 durante el sprint.

Estructura del repo (post ADR-0008)

```

argos/
├── README.md                # Entry point del proyecto
├── LICENSE                  # MIT
├── pyproject.toml           # Metadata + extras por módulo
├── .env.example             # Plantilla de variables (REAL .env va en .gitignore)
├── argos_contracts/        # ☐ Pydantic v2 cross-team (entregado · v1.1.0 · 69 tests)
│   ├── _mitre_data.py      # MITRE_WHITELIST curado
│   ├── alert.py            # WazuhAlert, NormalizedAlert
│   ├── ml_score.py         # MLScore (P2)
│   ├── triage.py           # AlertContext, TriageResponse (P1)
│   ├── incident.py        # Incident state machine (P1)
│   ├── approval.py        # ApprovalRequest, ApprovalResponse (P1)
│   ├── enums.py           # Tier, Severity, NotificationChannelType, ...
│   └── tests/              # 69 validation tests
├── llm_triage/             # P1 – Capa 4 LLM Triage
│   ├── api/main.py         # FastAPI /triage endpoint
│   ├── llm_client/         # OpenAI primary + Llama local fallback
│   ├── rag/                # BM25 + BGE-large + RRF
│   └── prompts/            # Jinja2 templates
├── soar/                   # P1 – SOAR Decision Engine + Approval API
│   ├── decision_engine/    # tier_classifier, state_machine, orchestrator
│   ├── approval/          # api, jwt_signer, consolidation
│   ├── notification/      # telegram, discord, twilio_voice, email
│   └── playbooks/          # host_isolation, process_kill, snapshot, throttle
├── ml/                     # P2 – Capa 2 ML
│   ├── features/           # Feature extractors por dominio
│   ├── models/             # ransomware_ensemble, network_traffic, query_pattern
│   └── consumer/           # Redis stream consumer
├── detection/              # P3 – Capa 1 Sigma rules
│   ├── sigma-rules/
│   │   ├── ransomware/    # T1486, T1490, T1083, T1562
│   │   ├── network/      # T1498, T1499 rate-based
│   │   ├── database/     # query patterns
│   │   └── webapp/        # T1190 SQLi signatures
│   └── wazuh-rules/       # Convertidas con sigma-cli
├── deception/             # P3 – Capa 3 Canary
│   ├── canary_generator.py # Genera canary files
│   └── fim-configs/        # Wazuh FIM rules
├── ui/                     # P4 (base) + P1 (Approval Console)
│   ├── streamlit_app/
│   │   ├── pages/
│   │   │   ├── 01_alert_inspection.py
│   │   │   ├── 02_approval_console.py # P1 - PIEZA CLAVE DEL DEMO
│   │   │   └── 03_audit_forensics.py
│   └── opensearch-dashboards/ # JSON dashboards exportados
└── attack-simulation/     # Simuladores multi-vector

```

```

├── ransomware_simulator/ # P1 (LockBit-like, canary, novel)
├── network_attacks/      # P4 (hping3, slowhttptest)
├── webapp_attacks/       # P4 (sqlmap)
├── pg_simulator/         # P4 (SELECT masivo para UC-07)
├── lab/                  # P4 – Vagrant + Terraform
│   ├── Vagrantfile
│   ├── provision/       # Scripts bash/PowerShell
│   └── inventory.yaml
├── evaluation/           # P2 + P4 – Métricas, datasets, reportes
└── docs/
    ├── PROJECT_BRIEF.md # 90-second overview
    ├── CONTEXT.md       # Onboarding completo
    ├── PROJECT_STATUS.md # Honest snapshot
    ├── EVALUATION_CRITERIA.md # Rúbrica del curso
    ├── data-handling.md  # T-030 sanitization policy
    ├── README.md         # Mapa de docs
    ├── architecture/     # SAD, threat model, flow diagrams
    ├── decisions/         # 8 ADRs + OPEN_QUESTIONS
    ├── use-cases/         # 8 UCs detallados
    └── team/              # Sprint manuals (este documento + 4 individuales)

```

Variables de entorno (.env)

El `.env.example` documenta TODAS las variables. Las críticas para tu rol están en tu manual individual. Reglas globales:

- **NUNCA** commitear el `.env` — está en `.gitignore`.
- Cada integrante genera su propio `.env` partir del `.env.example`.
- Las credenciales compartidas (Telegram bot token, Discord webhook URL, OpenAI API key) las distribuye P1 vía canal privado (no en GitHub, no en Discord público).
- El `JWT_SECRET` se genera localmente con `openssl rand -hex 32`.

Convención de incident IDs

`INC-YYYY-MM-DD-NNN` donde NNN es contador diario zero-padded a 3 dígitos. Ejemplo:
`INC-2026-05-26-001`. El contador se persiste en Redis con TTL diario.

10. Quick start del entorno común

Estos pasos son comunes a TODOS los integrantes antes de empezar el sprint. Tu manual individual asume que esto está hecho.

Paso 1 — Clonar repo

```
cd ~/projects # o donde guardes proyectos
git clone https://github.com/EnzoOrdonez/argos.git
cd argos
```

Paso 2 — Crear virtualenv Python

```
python3 -m venv .venv
source .venv/bin/activate # Linux/macOS
# .venv\Scripts\activate # Windows PowerShell
pip install -U pip setuptools wheel
```

Paso 3 — Instalar dependencias core + tu rol

```
# Todos instalan estos:
pip install -e ".[dev]"

# Además según tu rol:
pip install -e ".[contracts]" # P1 (siempre + más abajo)
pip install -e ".[ml]" # P2
pip install -e ".[soar,llm]" # P1 (servicios)
pip install -e ".[ui]" # P4
```

Paso 4 — Verificar contracts

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si no pasan los 69 tests, revisa que el venv esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 5 — Copiar plantilla de variables

```
cp .env.example .env
# Editar .env con tus valores reales (tu manual individual te dice cuáles)
```

Paso 6 — Configurar git

```
git config user.name "<Tu Nombre>"
git config user.email "<tu@email.com>"
# Crear tu branch:
git checkout -b feature/<tu-pX>/sprint-w1-init
```

11. Comunicación del equipo durante el sprint

Standup diario

- **Hora:** 9:00 AM
- **Canal:** Discord `#argos-standup`, llamada de voz
- **Duración:** 20 minutos (cada integrante ~3-4 min)
- **Formato** (per `docs/team/standup-template.md`):
 - Lo que cerré ayer (PRs mergeados, tests passing)
 - Lo que cierro hoy (objetivos del día)
 - Bloqueos (qué necesito de otro integrante)

Canales Discord del equipo

- `#argos-standup` — standup diario
- `#argos-help` — preguntas técnicas urgentes
- `#argos-prs` — notificación de PRs abiertos (GitHub bot)
- `#argos-incidents` — donde el bot ARGOS enviará notificaciones del demo

Reglas operativas (post ADR-0008)

1. **No tocar la doc esta semana.** Los docs están sincronizados con la realidad. Si descubres algo arquitectónico nuevo, abre un ADR-0009 en lugar de modificar SAD/threat model/README.
2. **No optimizar prematuramente.** El objetivo es que los 8 UCs corran end-to-end, no que sean óptimos. Performance fine-tuning en semana 2.
3. **Mocks son OK al inicio.** Si tu pieza depende de otra que aún no está lista, usa FakeRedis, mocked HTTP, o synthetic data. Integración real cuando ambas piezas estén listas.
4. **Verificar en lab real al menos 2 veces al día.** Cada integrante corre el flow E2E en su Vagrant antes del almuerzo y antes de pushear.
5. **Pedir ayuda en menos de 30 minutos.** Si llevas más de media hora atascado, pingueas en `#argos-help` o llamas a otro integrante. No debug solitario.
6. **Push diario obligatorio** antes de las 22:00. Si no pusheas, tu nombre aparece en el standup del día siguiente.

Claude Code / asistentes IA — política reinterpretada (ADR-0008)

Cada integrante puede usar Claude Code (o cualquier asistente IA: Cursor, GitHub Copilot, ChatGPT) como **acelerador en SU propia parte**. La condición no-negociable: cada integrante DEBE entender el código que produce con asistencia de IA y poder defenderlo en viva sin la asistencia presente. La asistencia es para velocidad, NO para reemplazar comprensión.

La regla específica que se mantiene: P1 (Enzo) no escribe código de otros integrantes con Claude Code. P1 puede asesorar y revisar, no producir el código de otros.

12. Lo que NO se entrega en esta semana 1

Para que no te sorprendas cuando llegue Día 7 sin estas cosas:

- **Calibración Q5 protocol** con dataset etiquetado real (~100 ransomware + ~500 benignas). Semana 2.
- **UC-03 split-brain con 4 aprobadores reales** no scripted. Semana 2-3.
- **UC-05 stealth attack** end-to-end pulido. Semana 2.
- **PRs Sigma upstream aceptados** por SigmaHQ maintainers (depende de tiempos externos). Semana 2-3.
- **Video demo final editado**. Semana 3.
- **Informe técnico final**. Semana 3.
- **Cross-encoder reranker en RAG** (descartado del scope v1 per ADR-0001 v2).

Lo que SÍ se completa al cierre del sprint Día 7: 5 UCs corriendo end-to-end (UC-01, UC-02, UC-04, UC-06, UC-07), con UC-08 como nice-to-have, con dos rehearsals al domingo.

13. Documentos relacionados (referencia completa)

Si algo de este documento no quedó claro o necesitas más profundidad:

Si quieres saber...	Lee
Arquitectura completa de cada bloque	docs/architecture/SOLUTION_ARCHITECTURE_DOCUMENT.md
Por qué se tomó cada decisión arquitectónica	docs/decisions/ (ADRs 0001-0008)
Análisis de amenazas STRIDE/FMEA	docs/architecture/THREAT_MODEL.md
Detalle de los 8 UCs	docs/use-cases/USE_CASES.md
Política de manejo de datos sensibles a LLM	docs/data-handling.md
Rúbrica del curso y mapping a deliverables	docs/EVALUATION_CRITERIA.md
Estado real vs. documentado del proyecto	docs/PROJECT_STATUS.md
Flujo del sistema visualmente	docs/architecture/argos_flow.html
Tu manual de sprint individual	docs/team/sprint-week-1-p<X>-<nombre>.md
Plan operacional general del sprint	docs/team/sprint-week-1-overview.md

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial — sección común de introducción para los 4 manuales individuales. Cubre arquitectura, UCs, ejemplo end-to-end UC-01, glosario MITRE, stack tecnológico, convenciones del repo, política Claude Code, quick start.	P1 (Enzo Ordoñez Flores)

Parte II — Manual individual: Enzo Ordoñez Flores

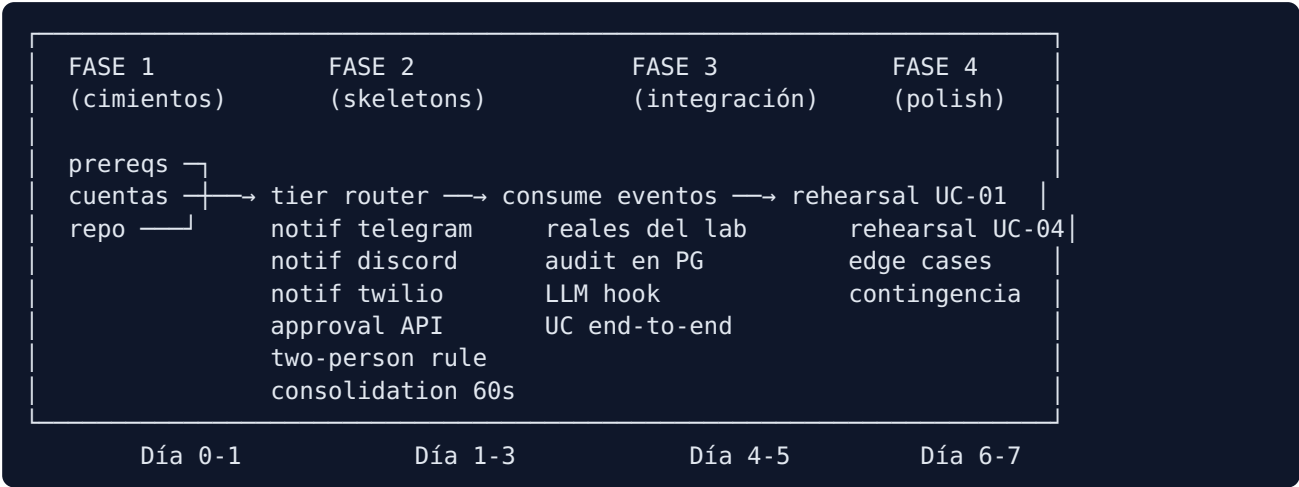
Manual P1 — Enzo Ordoñez Flores (Líder Técnico)

Campo	Valor
Rol	Líder técnico + integrador cross-layer
Owns	SOAR Decision Engine (<code>soar/</code>) · HITL workflow · Notification Service · Approval API · <code>argos_contracts</code>
No owns	Layers 1/3 (P3) · Layer 2 ML / Layer 4 LLM (P2) · Lab + UI + DB (P4)
Outputs blocking otros	<code>Incident</code> model en Redis → P4 (UI lee) · Tier router + Approver workflow → demo UC-04
Deadline	2026-06-13 (sábado) — demo en vivo + informe entregado
Cómo leer este manual	Linealmente. Cada fase asume la anterior cerrada. Cada sección termina con un checklist; no avances si algún check falla.

0. Tu charter en una frase

Tú haces que las 4 capas hablen entre ellas, que un humano pueda aprobar/rechazar acciones críticas, y que las notificaciones lleguen a 4 canales sin caerse. Si tu pieza falla, el demo falla. Si tu pieza funciona, el resto del equipo brilla.

0.1 Tu camino crítico (visual)



Las "fases" no son calendario rígido; son **orden de implementación**. Si terminas Fase 2 en día 2, salta a Fase 3. Si te trabas en Fase 2 hasta día 4, recortas Fase 4 y avisas en standup.

0.2 Cómo leer los outputs esperados

Cuando veas un bloque marcado **# OUTPUT ESPERADO:** debajo de un comando, ese es el output **literal** que debes ver, salvo timestamps y UUIDs. Si tu output difiere en estructura (no en valores numéricos), algo está mal. **No avances** hasta resolverlo: usa el Apéndice A (Troubleshooting) o pinguea en **#argos-help** antes de 30 minutos.

0.3 Convenciones de error handling

Cada sección tiene tres niveles de control:

1. **Output esperado inmediato** después de cada comando crítico.
2. **Checklist de verificación** al final de cada sección.
3. **Tabla de troubleshooting** en Apéndice A — entrada por síntoma observable.

Si los tres niveles te fallan, asume que es un bug del manual y abre issue (**bug: manual P1 sección X.Y**) en GitHub. **No pivotees el diseño sin aviso** — eso es lo que rompió a equipos de años anteriores.

FASE 1 — Cimientos

Goal: terminar Fase 1 en ≤ 8 horas de trabajo activo (típicamente día 0 noche + día 1 mañana). Si te lleva más, escala.

1.1 Verificar prerequisites del sistema

Qué estás haciendo

Confirmando que tu laptop tiene las versiones correctas de Python, git, Docker, y herramientas CLI antes de tocar el código. **El 80% de los errores raros vienen de versiones desalineadas** entre integrantes.

Comandos

```
# Versión de Python (requerimos 3.11.x exacto)
python3 --version
# OUTPUT ESPERADO:
# Python 3.11.7 (o cualquier 3.11.x; NO 3.10, NO 3.12)

# Versión de pip (>= 23.0)
pip --version
# OUTPUT ESPERADO:
# pip 23.x.x from /usr/lib/python3/dist-packages/pip (python 3.11)

# git instalado
git --version
# OUTPUT ESPERADO:
# git version 2.34.1 (cualquier >= 2.30)

# Docker (necesario para Wazuh local + OpenSearch + Redis)
docker --version && docker compose version
# OUTPUT ESPERADO:
# Docker version 24.x.x, build xxxxx
# Docker Compose version v2.x.x

# redis-cli (para inspeccionar incidents en Redis directo)
redis-cli --version
# OUTPUT ESPERADO:
# redis-cli 7.0.x (instalable con: apt install redis-tools)

# curl y jq (para probar APIs HTTP)
curl --version | head -1
jq --version
# OUTPUT ESPERADO:
# curl 7.81.0 (x86_64-pc-linux-gnu) ...
# jq-1.6
```

Comprobar que todo salió bien

Check	Comando	Esperado
Python 3.11.x	<code>python3 --version</code>	<code>3.11.x</code>
pip >= 23	<code>pip --version</code>	<code>23.x</code> o mayor
git >= 2.30	<code>git --version</code>	<code>>= 2.30</code>
Docker daemon corre	<code>docker ps</code>	tabla vacía sin error
redis-cli	<code>redis-cli --version</code>	<code>7.x</code>

Si **cualquiera falla** → Apéndice A.1 (instalación de prerequisites por OS).

1.2 Crear cuentas externas

Qué estás haciendo

ARGOS depende de 4 servicios externos. Crea cuentas ahora; pegarlas en `.env` después. Cada cuenta es **gratuita** en su plan de prueba.

1.2.1 Telegram bot (canal primario)

```
# Abre Telegram en tu celular o desktop
# Busca: @BotFather
# Envía: /newbot
# Nombre: ARGOS Alerts Bot
# Username: argos_alerts_<tu_inicial>_bot   (debe terminar en _bot)
#
# BotFather responde con:
# Use this token to access the HTTP API:
# 7123456789:AAFxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
#
# COPIA ESE TOKEN – es tu TELEGRAM_BOT_TOKEN.

# Para obtener tu chat_id (donde llegarán las alertas):
# 1. Envía cualquier mensaje a tu bot (hola)
# 2. Abre en navegador:
#   https://api.telegram.org/bot<TU_TOKEN>/getUpdates
# 3. En la respuesta JSON, busca: "chat":{"id": 123456789, ...}
# Ese número es tu TELEGRAM_CHAT_ID.

# Verifica que funciona:
curl -s -X POST "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/sendMessage" \
  -d "chat_id=${TELEGRAM_CHAT_ID}" \
  -d "text=ARGOS test from $(hostname)" | jq .ok
# OUTPUT ESPERADO:
# true
```

1.2.2 Discord webhook (canal secundario)

1. Crea un servidor Discord nuevo: "ARGOS Demo"
2. Crea canal #argos-alerts
3. Settings (engranaje del canal) → Integrations → Webhooks → New Webhook
4. Nombre: ARGOS Notifier
5. Copy Webhook URL → es tu DISCORD_WEBHOOK_URL
Formato: `https://discord.com/api/webhooks/<id>/<token>`

Verificar:

```
curl -X POST "$DISCORD_WEBHOOK_URL" \  
-H "Content-Type: application/json" \  
-d '{"content": "ARGOS webhook test"}'
```

OUTPUT ESPERADO:

(sin output – código HTTP 204; el mensaje aparece en el canal Discord)

1.2.3 Twilio (escalación T2 por voz DTMF)

1. <https://www.twilio.com/try-twilio> → cuenta trial (gratis, USD 15 credit)
2. Verificar tu celular como "verified caller" (Twilio trial restringe llamadas solo a números verificados – esto está OK para el demo).
3. Console → Account → API keys & tokens → copy:
 - Account SID → TWILIO_ACCOUNT_SID
 - Auth Token → TWILIO_AUTH_TOKEN
4. Get a Twilio phone number (trial te da uno gratis, USA)
→ TWILIO_FROM_NUMBER (formato +1XXXXXXXXXX)

Limitación trial: si los 4 integrantes están en Perú, Twilio puede no entregar llamadas internacionales en trial. **Plan B** ya documentado en ADR-0007 v2: Twilio se prueba en sandbox; si en el día del demo no funciona, escalación T2 cae automáticamente a Telegram + Discord con prefijo **[T2-ESCALATION]** .

1.2.4 OpenAI API key (LLM Triage, Layer 4)

1. <https://platform.openai.com/api-keys>
2. Create new secret key → nombre: "ARGOS demo"
3. Copy key (empieza con sk-proj-...) → OPENAI_API_KEY
4. Set billing limit a USD 5/mes en <https://platform.openai.com/account/billing/limits>
(gpt-4o-mini cuesta ~\$0.15 input + \$0.60 output por 1M tokens; budget sobrado para el demo + rehearsals.)

Fallback: si OpenAI cae, P2 tiene `LLM_BACKEND=llama_local` apuntando a Ollama corriendo Llama 3.1 8B en localhost. Tu trabajo aquí es **solo** crear la cuenta y poner la key; el switch lo hace P2 desde su capa.

Comprobar que todo salió bien

Check	Comando / Acción	Esperado
Bot Telegram responde	curl sendMessage con token	<code>"ok": true</code>
Webhook Discord funciona	curl POST	mensaje aparece en canal
Twilio credenciales válidas	login en console.twilio.com	dashboard carga
OpenAI key válida	<code>curl https://api.openai.com/v1/models -H "Authorization: Bearer \$OPENAI_API_KEY"</code>	JSON con lista de modelos (no <code>"invalid_api_key"</code>)

Si alguna falla: no pases a 1.3. Resolver antes — los 4 son críticos para el demo.

1.3 Clonar repo y preparar entorno Python

Comandos

```
# Carpeta de trabajo
mkdir -p ~/code && cd ~/code

# Clone
git clone git@github.com:enzizoor/argos.git
cd argos

# Verificar branch y estado
git status
# OUTPUT ESPERADO:
# On branch main
# Your branch is up to date with 'origin/main'.
# nothing to commit, working tree clean

# Crear virtualenv
python3 -m venv .venv
source .venv/bin/activate
# El prompt debe cambiar a:
# (.venv) usuario@host:~/code/argos$

# Actualizar pip dentro del venv
pip install --upgrade pip
# OUTPUT ESPERADO (última línea):
# Successfully installed pip-24.x.x

# Instalar argos_contracts en modo editable
pip install -e ./argos_contracts
# OUTPUT ESPERADO (últimas líneas):
# Successfully built argos_contracts
# Successfully installed argos_contracts-1.1.0 pydantic-2.x.x ...

# Verificar que el paquete importa
python -c "import argos_contracts; print(argos_contracts.__version__)"
# OUTPUT ESPERADO:
# 1.1.0

# Instalar dependencias del SOAR
pip install -r soar/requirements.txt
# OUTPUT ESPERADO (última línea):
# Successfully installed fastapi-0.110.x uvicorn-0.27.x redis-5.0.x httpx-0.27.x ...
```

Crear `.env` con tus credenciales

```
# El archivo .env NUNCA se commitea (está en .gitignore).
# Plantilla:
cat > .env << 'EOF'
# === Telegram ===
TELEGRAM_BOT_TOKEN=7123456789:AAFxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TELEGRAM_CHAT_ID=123456789

# === Discord ===
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/.../...

# === Twilio ===
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_FROM_NUMBER=+1XXXXXXXXXX
TWILIO_TO_NUMBER=+51XXXXXXXXXX # tu número verificado

# === OpenAI ===
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
LLM_BACKEND=openai_gpt4o_mini # alternativa: llama_local

# === Redis (lab Vagrant local) ===
REDIS_URL=redis://localhost:6379/0

# === PostgreSQL (audit log) ===
ARGOS_PG_URL=postgresql://argos:argos@localhost:5432/argos_audit
EOF

# Reemplaza los placeholders con tus valores reales.
chmod 600 .env
```


Comprobar que todo salió bien

```
# Test smoke
pytest -q
# OUTPUT ESPERADO:
# ..... [100%]
# 69 passed, 1 warning in 0.27s

# Confirmar venv activo y deps instaladas
which python
# OUTPUT ESPERADO:
# /home/<usuario>/code/argos/.venv/bin/python

pip list | grep -E "(argos|fastapi|redis|pydantic)"
# OUTPUT ESPERADO (4 líneas, versiones aproximadas):
# argos-contracts      1.1.0
# fastapi               0.110.x
# pydantic              2.x.x
# redis                 5.0.x
```

Check Fase 1	Estado esperado
<code>python3 --version</code>	3.11.x
Tests existentes pasan	69 passed
<code>.env</code> existe con 4 credenciales reales	sí
Telegram, Discord, OpenAI keys funcionan	sí (verificado en 1.2)
<code>git status</code>	working tree clean

No avances a Fase 2 si cualquier check de 1.3 está rojo.

FASE 2 — Skeletons funcionales

*Goal: cada componente individual corre y pasa sus tests unitarios, **aún con stubs** en lugar de servicios reales. Conexión al lab real viene en Fase 3.*

2.1 SOAR Decision Engine — Tier Router

Qué estás haciendo

El Tier Router es **la pieza más crítica de todo ARGOS**: recibe un evento normalizado de cualquier capa y decide qué Tier asignar (T0 / T1 / T2 / T3). De ese tier salen las acciones automáticas y/o la solicitud de aprobación humana.

Estructura de archivos

```
soar/
├─ requirements.txt
├─ decision_engine/
│   ├── __init__.py
│   ├── tier_router.py    ← tú escribes
│   ├── policies.py      ← tú escribes (matrices de tier)
│   └─ tests/
│       ├── test_tier_router.py
│       └─ test_policies.py
```

Crear `tier_router.py` (código central completo)

```

# soar/decision_engine/tier_router.py
"""
Tier Router – asigna T0/T1/T2/T3 a un evento normalizado.

Regla:
- T0: alta confianza, acción inmediata automática (aislar host).
- T1: alta confianza, acción automática reversible (quarantine).
- T2: confianza media, requiere aprobación humana (HITL).
- T3: baja confianza, sólo log + dashboard.

El tier sale de combinar:
- severity (de la regla disparada)
- num_layers_fired (1 capa = más débil; 3+ = más fuerte)
- confidence_score (ML / LLM)
- mitre_technique (algunos son auto-T0 por política)
"""

from __future__ import annotations

from typing import Literal

from argos_contracts.enums import Tier
from argos_contracts.incident import NormalizedEvent
from soar.decision_engine.policies import (
    AUTO_T0_TECHNIQUES,
    MIN_LAYERS_FOR_AUTO,
    SEVERITY_T0_BASE_Tier,
)

ConfidenceBand = Literal["high", "medium", "low"]

def confidence_band(score: float) -> ConfidenceBand:
    """Map [0.0, 1.0] → high/medium/low."""
    if score >= 0.85:
        return "high"
    if score >= 0.55:
        return "medium"
    return "low"

def route(event: NormalizedEvent) -> Tier:
    """
    Devuelve el Tier asignado a un evento. Pura: no I/O, no mutación.

    Args:
        event: evento normalizado proveniente de cualquier capa.

    Returns:
        Tier (T0, T1, T2, o T3).
    """
    # 1. Auto-T0 por técnica MITRE crítica (ransomware encryption, mass delete)
    if event.mitre_technique in AUTO_T0_TECHNIQUES:
        return Tier.T0

```

```

# 2. Tier base por severity
base = SEVERITY_T0_BASE_TIER[event.severity]

# 3. Boost por número de capas que coincidieron
if event.num_layers_fired >= MIN_LAYERS_FOR_AUTO:
    # 3+ capas → forzar a T0 si base ≤ T1
    if base in (Tier.T1, Tier.T2):
        return Tier.T0

# 4. Down-tier si confianza baja (single layer + low confidence)
band = confidence_band(event.confidence_score)
if event.num_layers_fired == 1 and band == "low":
    return Tier.T3

return base

```

Crear `policies.py` (matrices completas)

```

# soar/decision_engine/policies.py
"""
Matrices y constantes de política para el Tier Router.

Cualquier cambio aquí cambia el comportamiento del demo. Mantener una sola fuente
de verdad evita branchear lógica en múltiples archivos.
"""

from argos_contracts.enums import Severity, Tier

# Cualquier técnica en esta lista dispara T0 automático (acción inmediata).
# Justificación: el costo de un FP es bajo (aislar un host), el costo de un FN
# es altísimo (ransomware completo).
AUTO_T0_TECHNIQUES: frozenset[str] = frozenset({
    "T1486", # Data Encrypted for Impact (ransomware)
    "T1490", # Inhibit System Recovery (delete shadow copies)
    "T1485", # Data Destruction
    "T1561", # Disk Wipe
})

# A partir de cuántas capas coincidentes consideramos "consenso" → boost a T0.
MIN_LAYERS_FOR_AUTO: int = 3

# Mapeo base severity → Tier antes de aplicar boosts.
SEVERITY_T0_BASE_TIER: dict[Severity, Tier] = {
    Severity.CRITICAL: Tier.T0,
    Severity.HIGH: Tier.T1,
    Severity.MEDIUM: Tier.T2,
    Severity.LOW: Tier.T3,
}

```

Tests unitarios completos

```

# soar/decision_engine/tests/test_tier_router.py
import pytest

from argos_contracts.enums import Severity, Tier
from argos_contracts.incident import NormalizedEvent
from soar.decision_engine.tier_router import route, confidence_band

def make_event(**overrides) -> NormalizedEvent:
    """Factory con defaults sensatos para tests."""
    base = dict(
        event_id="evt-test-001",
        severity=Severity.MEDIUM,
        mitre_technique="T1083", # File and Directory Discovery
        num_layers_fired=1,
        confidence_score=0.7,
        host="WIN-VICTIM-01",
        layer_origin="sigma",
    )
    base.update(overrides)
    return NormalizedEvent(**base)

# --- confidence_band -----
@pytest.mark.parametrize("score, expected", [
    (0.95, "high"), (0.85, "high"),
    (0.84, "medium"), (0.60, "medium"), (0.55, "medium"),
    (0.54, "low"), (0.10, "low"),
])
def test_confidence_band(score, expected):
    assert confidence_band(score) == expected

# --- route -----
def test_auto_t0_for_ransomware_technique():
    e = make_event(mitre_technique="T1486", severity=Severity.LOW)
    assert route(e) == Tier.T0 # técnica crítica fuerza T0 aún con severity LOW

def test_critical_severity_maps_to_t0():
    e = make_event(severity=Severity.CRITICAL, mitre_technique="T1083")
    assert route(e) == Tier.T0

def test_three_layers_boost_t2_to_t0():
    e = make_event(severity=Severity.MEDIUM, num_layers_fired=3)
    assert route(e) == Tier.T0

def test_single_layer_low_confidence_drops_to_t3():
    e = make_event(severity=Severity.MEDIUM, num_layers_fired=1, confidence_score=0.3)
    assert route(e) == Tier.T3

```

```
def test_medium_severity_two_layers_stays_t2():
    e = make_event(severity=Severity.MEDIUM, num_layers_fired=2, confidence_score=0.7)
    assert route(e) == Tier.T2
```

Correr y verificar

```
# Desde la raíz del repo, con venv activo:
pytest soar/decision_engine/tests/ -v
# OUTPUT ESPERADO:
# soar/decision_engine/tests/test_tier_router.py::test_confidence_band[0.95-high] PASSED
# soar/decision_engine/tests/test_tier_router.py::test_confidence_band[0.85-high] PASSED
# ... (7 parametrize + 5 tests = 12 PASSED)
# ===== 12 passed in 0.08s =====
```

Comprobar que todo salió bien (sección 2.1)

Check	Comando	Esperado
Tests router	<code>pytest soar/decision_engine/tests/test_tier_router.py -v</code>	12 passed
Sin imports rotos	<code>python -c "from soar.decision_engine.tier_router import route; print('ok')"</code>	ok
Coverage del router ≥ 90%	<code>pytest --cov=soar.decision_engine.tier_router soar/</code>	≥90%

2.2 Notification Service — Estructura base

Qué estás haciendo

Adapter pattern: una interfaz `NotificationChannel` con N implementaciones (Telegram, Discord, Twilio, Email). El `NotificationService` recibe un `Incident` y despacha a los canales correspondientes según política.

Estructura

```
soar/
├── notifications/
│   ├── __init__.py
│   ├── base.py           ← interfaz + clase base (full)
│   ├── service.py       ← orquestador (full)
│   └── channels/
│       ├── __init__.py
│       ├── telegram.py   ← 2.3 (full)
│       ├── discord.py    ← 2.4 (full)
│       ├── twilio_voice.py ← 2.5 (full)
│       └── email_postfact.py ← skeleton + reference
└── tests/
    ├── test_service.py
    └── channels/
```

Crear `base.py` (interfaz central)

```
# soar/notifications/base.py
"""
Interfaz común a todos los canales de notificación.

Convención: un canal SIEMPRE devuelve DispatchResult, nunca tira excepción
hacia afuera. Eso permite al Service degradar (intentar canal B si canal A
falla) sin try/except esparcido.
"""

from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional

from argos_contracts.incident import Incident
from argos_contracts.enums import NotificationChannelType

@dataclass(frozen=True)
class DispatchResult:
    channel: NotificationChannelType
    success: bool
    latency_ms: int
    error: Optional[str] = None

class NotificationChannel(ABC):
    """Cada canal implementa esto. Sin estado mutable salvo `self.config`."""

    channel_type: NotificationChannelType # subclase setea como class var

    @abstractmethod
    def dispatch(self, incident: Incident) -> DispatchResult:
        """Envía notificación. Nunca tira excepción al caller."""
        ...
```

Crear `service.py` (orquestador central)

```

# soar/notifications/service.py
"""
Orquesta despacho a múltiples canales según el Tier del incidente.

Política de despacho (de ADR-0007 v2):
    T0 → Telegram + Discord (informativo, sin pedir aprobación)
    T1 → Telegram + Discord (informativo)
    T2 → Telegram + Discord (con botones de aprobación)
        + Twilio Voice si nadie respondió en 60s
    T3 → solo log (no notificación)
"""

from __future__ import annotations

import logging
import time
from typing import Iterable

from argos_contracts.enums import Tier, NotificationChannelType
from argos_contracts.incident import Incident

from soar.notifications.base import NotificationChannel, DispatchResult

logger = logging.getLogger(__name__)

# Política: qué canales para qué tier en el primer disparo.
TIER_CHANNELS: dict[Tier, list[NotificationChannelType]] = {
    Tier.T0: [NotificationChannelType.TELEGRAM, NotificationChannelType.DISCORD],
    Tier.T1: [NotificationChannelType.TELEGRAM, NotificationChannelType.DISCORD],
    Tier.T2: [NotificationChannelType.TELEGRAM, NotificationChannelType.DISCORD],
    Tier.T3: [],
}

class NotificationService:
    """
    Inyecta canales en el constructor → testeable con fakes.
    Las llamadas dispatch() son síncronas: para HITL no necesitamos throughput,
    necesitamos predictibilidad.
    """

    def __init__(self, channels: Iterable[NotificationChannel]):
        self._channels: dict[NotificationChannelType, NotificationChannel] = {
            c.channel_type: c for c in channels
        }

    def dispatch_for_tier(self, incident: Incident) -> list[DispatchResult]:
        wanted = TIER_CHANNELS.get(incident.tier, [])
        results: list[DispatchResult] = []
        for channel_type in wanted:
            channel = self._channels.get(channel_type)
            if channel is None:
                results.append(DispatchResult(

```

```

        channel=channel_type,
        success=False,
        latency_ms=0,
        error="channel not configured",
    ))
    continue
    t0 = time.monotonic()
    try:
        results.append(channel.dispatch(incident))
    except Exception as exc: # noqa: BLE001 – defensive último resorte
        logger.exception("channel %s raised unexpected exception", channel_type)
        results.append(DispatchResult(
            channel=channel_type,
            success=False,
            latency_ms=int((time.monotonic() - t0) * 1000),
            error=f"unexpected: {type(exc).__name__}: {exc}",
        ))
    return results

def escalate_to_voice(self, incident: Incident) -> DispatchResult:
    """T2 a t=60s sin respuesta → llamada Twilio (DTMF)."""
    voice = self._channels.get(NotificationChannelType.TWILIO_VOICE)
    if voice is None:
        return DispatchResult(
            channel=NotificationChannelType.TWILIO_VOICE,
            success=False, latency_ms=0,
            error="twilio not configured",
        )
    return voice.dispatch(incident)

```

Tests del orquestador (parte boilerplate, skeleton + referencia)

```

# soar/notifications/tests/test_service.py
"""
Verifica:
- dispatch_for_tier respeta TIER_CHANNELS
- Si un canal lanza excepción, se captura → success=False (no propaga)
- T3 no despacha nada

Patrón: FakeChannel implementa NotificationChannel devolviendo lo que le digamos.
Ver `tests/conftest.py` para fixture `fake_channel_factory`.
"""

# ... (tests parametrizados – patrón estándar pytest; ver
# docs/contracts/CONTRACTS_SPECIFICATION.md §3 para ejemplos similares)

```

Comprobar que todo salió bien (sección 2.2)

Check	Comando	Esperado
<code>base.py</code> y <code>service.py</code> importan	<code>python -c "from soar.notifications.service import NotificationService; print('ok')"</code>	<code>ok</code>
Tests del service pasan	<code>pytest soar/notifications/tests/test_service.py -v</code>	<code>>= 5 passed</code>
Cobertura del service $\geq$ 80%	<code>pytest --cov=soar.notifications.service</code>	<code>>=80%</code>

2.3 Notification Channel — Telegram

Código completo

```

# soar/notifications/channels/telegram.py
"""
Telegram bot – canal primario para todos los tiers != T3.

Mensajes formateados con MarkdownV2 (Telegram-flavored). Botones inline para T2
(Approve / Reject) cuando incident.requires_approval == True.
"""

from __future__ import annotations

import logging
import os
import time
from typing import Optional

import httpx

from argos_contracts.enums import NotificationChannelType, Tier
from argos_contracts.incident import Incident

from soar.notifications.base import DispatchResult, NotificationChannel

logger = logging.getLogger(__name__)

_API = "https://api.telegram.org/bot{token}/sendMessage"

def _escape_md(text: str) -> str:
    """MarkdownV2 reserved: _*[]()~`>#+-=|{.!"'"""
    for ch in r"_*[]()~`>#+-=|{.!\\"":
        text = text.replace(ch, f"\\{ch}")
    return text

def _format(incident: Incident) -> str:
    tier_emoji = {Tier.T0: "🔴", Tier.T1: "🟡", Tier.T2: "🟢", Tier.T3: "🟠"}
    return (
        f"{tier_emoji[incident.tier]} *ARGOS {incident.tier.value}* – "
        f"`{_escape_md(incident.host.hostname)}`\n"
        f"*Técnica:* `{_escape_md(incident.mitre_technique)}`\n"
        f"*Capas firing:* `{incident.num_layers_fired}`\n"
        f"*Confianza:* `{incident.confidence_score:.2f}`\n"
        f"*ID:* `{_escape_md(incident.incident_id)}"`
    )

def _inline_keyboard_for_approval(incident_id: str) -> dict:
    return {
        "inline_keyboard": [
            {
                "text": "👍 Approve", "callback_data": f"approve:{incident_id}",
                "text": "👎 Reject", "callback_data": f"reject:{incident_id}",
            }
        ]
    }

```



```

class TelegramChannel(NotificationChannel):
    channel_type = NotificationChannelType.TELEGRAM

    def __init__(self, bot_token: Optional[str] = None,
                  chat_id: Optional[str] = None,
                  client: Optional[httpx.Client] = None,
                  timeout: float = 5.0):
        self._token = bot_token or os.environ["TELEGRAM_BOT_TOKEN"]
        self._chat_id = chat_id or os.environ["TELEGRAM_CHAT_ID"]
        self._client = client or httpx.Client(timeout=timeout)

    def dispatch(self, incident: Incident) -> DispatchResult:
        t0 = time.monotonic()
        body = {
            "chat_id": self._chat_id,
            "text": _format(incident),
            "parse_mode": "MarkdownV2",
        }
        if incident.requires_approval:
            body["reply_markup"] = _inline_keyboard_for_approval(incident.incident_id)
        try:
            r = self._client.post(_API.format(token=self._token), json=body)
            r.raise_for_status()
            payload = r.json()
            if not payload.get("ok"):
                return DispatchResult(
                    channel=self.channel_type, success=False,
                    latency_ms=int((time.monotonic() - t0) * 1000),
                    error=str(payload.get("description")),
                )
            return DispatchResult(
                channel=self.channel_type, success=True,
                latency_ms=int((time.monotonic() - t0) * 1000),
            )
        except httpx.HTTPError as exc:
            return DispatchResult(
                channel=self.channel_type, success=False,
                latency_ms=int((time.monotonic() - t0) * 1000),
                error=f"http: {exc}",
            )

```

Prueba manual (smoke test)

```
# Exportar credenciales y simular un incidente T0
export $(grep -v '^#' .env | xargs)

python - << 'PY'
from argos_contracts.incident import Incident, HostInfo
from argos_contracts.enums import Tier, Severity, IncidentState
from soar.notifications.channels.telegram import TelegramChannel
from datetime import datetime, timezone

incident = Incident(
    incident_id="inc-smoke-001",
    tier=Tier.T0,
    state=IncidentState.NEW,
    severity=Severity.CRITICAL,
    host=HostInfo(hostname="WIN-VICTIM-01", ip="192.168.56.20", os="Windows 10"),
    mitre_technique="T1486",
    num_layers_fired=3,
    confidence_score=0.94,
    created_at=datetime.now(timezone.utc),
    requires_approval=False,
    approvers=[], final_decision=None, consolidation_window=None,
)
result = TelegramChannel().dispatch(incident)
print(result)
PY
# OUTPUT ESPERADO:
# DispatchResult(channel=<NotificationChannelType.TELEGRAM: 'telegram'>, success=True, latency_ms=28)
#
# Y en tu Telegram:
# 📧 ARGOS T0 – WIN-VICTIM-01
# Técnica: T1486
# Capas firing: 3
# Confianza: 0.94
# ID: inc-smoke-001
```

Comprobar que todo salió bien (sección 2.3)

Check	Esperado
Smoke test arriba devuelve <code>success=True</code>	sí
Mensaje llega a Telegram en < 1s	sí
Si pones token inválido, devuelve <code>success=False</code> con error visible (no excepción)	sí

2.4 Notification Channel — Discord

Código completo

```

# soar/notifications/channels/discord.py
from __future__ import annotations

import logging, os, time
from typing import Optional

import httpx

from argos_contracts.enums import NotificationChannelType, Tier
from argos_contracts.incident import Incident
from soar.notifications.base import DispatchResult, NotificationChannel

logger = logging.getLogger(__name__)

_COLOR = {Tier.T0: 0xE53935, Tier.T1: 0xFB8C00, Tier.T2: 0xFDD835, Tier.T3: 0x1E88E5}

def _embed(incident: Incident) -> dict:
    return {
        "title": f"ARGOS {incident.tier.value} - {incident.host.hostname}",
        "color": _COLOR[incident.tier],
        "fields": [
            {"name": "Técnica MITRE", "value": incident.mitre_technique, "inline": True},
            {"name": "Capas firing", "value": str(incident.num_layers_fired), "inline": True},
            {"name": "Confianza", "value": f"{incident.confidence_score:.2f}", "inline": True},
            {"name": "Incident ID", "value": f"`{incident.incident_id}`"},
        ],
        "footer": {"text": "ARGOS · respond on Telegram for approval"},
    }

class DiscordChannel(NotificationChannel):
    channel_type = NotificationChannelType.DISCORD

    def __init__(self, webhook_url: Optional[str] = None,
                 client: Optional[httpx.Client] = None,
                 timeout: float = 5.0):
        self._url = webhook_url or os.environ["DISCORD_WEBHOOK_URL"]
        self._client = client or httpx.Client(timeout=timeout)

    def dispatch(self, incident: Incident) -> DispatchResult:
        t0 = time.monotonic()
        body = {"embeds": [_embed(incident)]}
        try:
            r = self._client.post(self._url, json=body)
            # Discord webhook devuelve 204 No Content en éxito.
            if r.status_code in (200, 204):
                return DispatchResult(
                    channel=self.channel_type, success=True,
                    latency_ms=int((time.monotonic() - t0) * 1000),
                )
            return DispatchResult(
                channel=self.channel_type, success=False,
                latency_ms=int((time.monotonic() - t0) * 1000),
            )

```

```

        error=f"http {r.status_code}: {r.text[:200]}",
    )
except httpx.HTTPError as exc:
    return DispatchResult(
        channel=self.channel_type, success=False,
        latency_ms=int((time.monotonic() - t0) * 1000),
        error=f"http: {exc}",
    )

```

Smoke test

```

python - << 'PY'
import os
from datetime import datetime, timezone
from argos_contracts.incident import Incident, HostInfo
from argos_contracts.enums import Tier, Severity, IncidentState
from soar.notifications.channels.discord import DiscordChannel

i = Incident(
    incident_id="inc-smoke-002", tier=Tier.T2, state=IncidentState.NEW,
    severity=Severity.MEDIUM,
    host=HostInfo(hostname="LIN-VICTIM-01", ip="192.168.56.21", os="Ubuntu 22.04"),
    mitre_technique="T1083", num_layers_fired=2, confidence_score=0.71,
    created_at=datetime.now(timezone.utc), requires_approval=True,
    approvers=[], final_decision=None, consolidation_window=None,
)
print(DiscordChannel().dispatch(i))
PY
# OUTPUT ESPERADO:
# DispatchResult(channel=<NotificationChannelType.DISCORD: 'discord'>, success=True, latency_ms=312,
#
# En Discord aparece un embed amarillo con los 4 fields.

```

Comprobar (sección 2.4)

Check	Esperado
Smoke test devuelve <code>success=True</code>	sí
Embed visible en canal #argos-alerts con color correcto por tier	sí
URL mal escrita → <code>success=False</code> con error 401/404, no excepción	sí

2.5 Notification Channel — Twilio Voice (escalación T2)

Por qué importa

Sólo se dispara para T2 a t=60s sin respuesta. La llamada lee TwiML que enuncia el incidente y captura DTMF (**1=approve** , **2=reject**). Se llama **una vez por aprobador** que no haya respondido. Es el último recurso antes de que el conservative-wins policy decida solo.

Código completo

```

# soar/notifications/channels/twilio_voice.py
from __future__ import annotations

import logging, os, time
from typing import Optional
from urllib.parse import urlencode

import httpx

from argos_contracts.enums import NotificationChannelType
from argos_contracts.incident import Incident
from soar.notifications.base import DispatchResult, NotificationChannel

logger = logging.getLogger(__name__)

def _twiml_url(incident_id: str, base: str) -> str:
    """URL pública que sirve el TwiML.

    Para el lab usamos `ngrok` apuntando al Approval API local.
    El TwiML responde:
    <Response>
      <Gather numDigits="1" action="/voice/dtmf?incident=...">
        <Say voice="alice">ARGOS critical incident on host X.
          Press 1 to approve, 2 to reject.</Say>
      </Gather>
    </Response>
    """
    return f"{base}/voice/twiml?{urlencode({'incident': incident_id})}"

class TwilioVoiceChannel(NotificationChannel):
    channel_type = NotificationChannelType.TWILIO_VOICE

    def __init__(self,
        account_sid: Optional[str] = None,
        auth_token: Optional[str] = None,
        from_number: Optional[str] = None,
        to_number: Optional[str] = None,
        public_base_url: Optional[str] = None,
        client: Optional[httpx.Client] = None,
        timeout: float = 8.0):
        self._sid = account_sid or os.environ["TWILIO_ACCOUNT_SID"]
        self._tok = auth_token or os.environ["TWILIO_AUTH_TOKEN"]
        self._from = from_number or os.environ["TWILIO_FROM_NUMBER"]
        self._to = to_number or os.environ["TWILIO_TO_NUMBER"]
        self._base = public_base_url or os.environ.get("ARGOS_PUBLIC_URL", "")
        self._client = client or httpx.Client(timeout=timeout,
            auth=(self._sid, self._tok))

    def dispatch(self, incident: Incident) -> DispatchResult:
        t0 = time.monotonic()
        url = f"https://api.twilio.com/2010-04-01/Accounts/{self._sid}/Calls.json"
        try:

```



```

        r = self._client.post(url, data={
            "From": self._from,
            "To": self._to,
            "Url": _twiml_url(incident.incident_id, self._base),
            "Method": "POST",
            "Timeout": "20",
        })
    if r.status_code in (200, 201):
        return DispatchResult(
            channel=self.channel_type, success=True,
            latency_ms=int((time.monotonic() - t0) * 1000),
        )
    return DispatchResult(
        channel=self.channel_type, success=False,
        latency_ms=int((time.monotonic() - t0) * 1000),
        error=f"http {r.status_code}: {r.text[:300]}",
    )
except httpx.HTTPError as exc:
    return DispatchResult(
        channel=self.channel_type, success=False,
        latency_ms=int((time.monotonic() - t0) * 1000),
        error=f"http: {exc}",
    )

```

Smoke test (cuidado — gasta credit Twilio trial)

```
# Solo cuando estés listo para probar. Cada llamada consume ~$0.013 trial credit.
# El Approval API debe estar corriendo (sección 2.6) y ngrok debe estar levantado:
# ngrok http 8001

export ARGOS_PUBLIC_URL=https://abcd-1234.ngrok-free.app

python - << 'PY'
import os
from datetime import datetime, timezone
from argos_contracts.incident import Incident, HostInfo
from argos_contracts.enums import Tier, Severity, IncidentState
from soar.notifications.channels.twilio_voice import TwilioVoiceChannel

i = Incident(
    incident_id="inc-smoke-003", tier=Tier.T2, state=IncidentState.NEW,
    severity=Severity.MEDIUM,
    host=HostInfo(hostname="WIN-VICTIM-01", ip="192.168.56.20", os="Windows 10"),
    mitre_technique="T1083", num_layers_fired=2, confidence_score=0.72,
    created_at=datetime.now(timezone.utc), requires_approval=True,
    approvers=[], final_decision=None, consolidation_window=None,
)
print(TwilioVoiceChannel().dispatch(i))
PY
# OUTPUT ESPERADO:
# DispatchResult(channel=<NotificationChannelType.TWILIO_VOICE: 'twilio_voice'>, success=True, latency=...)
#
# Y en tu celular: una llamada de tu número Twilio. Si contestas, una voz
# dice "ARGOS critical incident...". Presionas 1 o 2.
```

Comprobar (sección 2.5)

Check	Esperado
Trial credit > \$1 antes de pruebas	sí
<code>ngrok http 8001</code> corriendo (para el TwiML callback)	sí
<code>success=True</code> y la llamada entra al celular	sí
Si <code>TWILIO_TO_NUMBER</code> no está verificado → error 21219 visible en <code>result.error</code>	sí

2.6 Approval API — FastAPI endpoints

Qué estás haciendo

Una mini API HTTP que recibe respuestas de aprobación de **tres orígenes**:

- Telegram → callback de botón inline → webhook en `/telegram/callback`
- Discord → reacción/comando slash → webhook en `/discord/callback` (*opcional para v1*)
- Twilio → DTMF gather → POST a `/voice/dtmf`

Cualquier respuesta mutiá el `Incident` en Redis y dispara la evaluación de la regla two-person.

Estructura

```
soar/approval_api/
├── __init__.py
├── main.py           ← FastAPI app + endpoints (full)
├── handlers.py       ← lógica HITL (two-person, consolidation) (full)
├── twiml.py          ← genera XML para Twilio (full)
├── tests/
│   ├── test_main.py
│   ├── test_handlers.py
│   └── test_twiml.py
```

`main.py` (entrypoint completo)

```

# soar/approval_api/main.py
"""
Approval API – recibe respuestas de los aprobadores y muta el Incident en Redis.

NO escribe acciones (aislar host, kill proceso, etc.); eso lo hace el SOAR
Decision Engine al ver el final_decision en Redis. Esta API es solo el plano
de control HITL.
"""

from __future__ import annotations

import os
from contextlib import asynccontextmanager
from typing import AsyncIterator

import redis.asyncio as redis
from fastapi import FastAPI, Form, HTTPException, Response

from soar.approval_api.handlers import (
    record_approval_response,
    build_final_decision_if_ready,
)
from soar.approval_api.twiml import build_voice_gather_xml, dtmf_to_response

@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    """Conexión Redis compartida en app.state."""
    app.state.redis = redis.from_url(
        os.environ["REDIS_URL"], decode_responses=True
    )
    try:
        yield
    finally:
        await app.state.redis.close()

app = FastAPI(title="ARGOS Approval API", lifespan=lifespan)

@app.get("/healthz")
async def healthz() -> dict:
    pong = await app.state.redis.ping()
    return {"ok": True, "redis": pong}

# ----- Telegram callback -----
@app.post("/telegram/callback")
async def telegram_callback(update: dict) -> dict:
    """
    Telegram Bot API envía updates aquí cuando alguien toca un botón inline.
    Estructura esperada:
        update["callback_query"] = {
            "id": "...", "data": "approve:inc-001",

```

```

        "from": {"id": 12345, "first_name": "Enzo"},
    }
    """
    cq = update.get("callback_query")
    if cq is None:
        raise HTTPException(400, "missing callback_query")

    action, _, incident_id = (cq.get("data") or "").partition(":")
    if action not in ("approve", "reject") or not incident_id:
        raise HTTPException(400, "bad callback_data")

    user = cq.get("from", {})
    approver_id = f"telegram:{user.get('id')}"

    await record_approval_response(
        redis_client=app.state.redis,
        incident_id=incident_id,
        approver_id=approver_id,
        decision=action,
        channel="telegram",
    )
    await build_final_decision_if_ready(app.state.redis, incident_id)
    return {"ok": True}

# ----- Twilio voice -----
@app.post("/voice/twiml", response_class=Response)
async def voice_twiml(incident: str) -> Response:
    """Devuelve TwiML que Twilio reproduce; el Gather captura DTMF."""
    xml = build_voice_gather_xml(incident_id=incident)
    return Response(content=xml, media_type="application/xml")

@app.post("/voice/dtmf", response_class=Response)
async def voice_dtmf(Digits: str = Form(...), incident: str = Form(...)) -> Response:
    decision = dtmf_to_response(Digits)
    if decision is None:
        return Response(
            content="<Response><Say>Invalid input. Bye.</Say><Hangup/></Response>",
            media_type="application/xml",
        )
    await record_approval_response(
        redis_client=app.state.redis,
        incident_id=incident,
        approver_id=f"twilio:{incident}",
        decision=decision,
        channel="twilio_voice",
    )
    await build_final_decision_if_ready(app.state.redis, incident)
    return Response(
        content=f"<Response><Say>{decision} recorded. Goodbye.</Say><Hangup/></Response>",
        media_type="application/xml",
    )

```

twiml.py (helper completo, pequeño)

```
# soar/approval_api/twiml.py
"""Helpers para generar TwiML y traducir DTMF."""

from typing import Optional, Literal

Decision = Literal["approve", "reject"]

def build_voice_gather_xml(incident_id: str) -> str:
    return f"""<?xml version="1.0" encoding="UTF-8"?>
<Response>
  <Gather numDigits="1" action="/voice/dtmf" method="POST" timeout="20">
    <Say voice="alice">
      ARGOS critical incident. Incident {incident_id}.
      Press one to approve, two to reject.
    </Say>
    <Pause length="2"/>
  </Gather>
  <Say>No input received. Goodbye.</Say>
  <Hangup/>
</Response>"""

_DTMF_TO_DECISION: dict[str, Decision] = {"1": "approve", "2": "reject"}

def dtmf_to_response(digits: str) -> Optional[Decision]:
    return _DTMF_TO_DECISION.get(digits.strip()) if digits else None
```

Correr y verificar

```
# Levantar la API en otra terminal:
uvicorn soar.approval_api.main:app --host 0.0.0.0 --port 8001 --reload
# OUTPUT ESPERADO:
# INFO:      Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
# INFO:      Started reloader process [12345] using StatReload
# INFO:      Started server process [12346]
# INFO:      Waiting for application startup.
# INFO:      Application startup complete.

# Health check
curl -s http://localhost:8001/healthz | jq
# OUTPUT ESPERADO:
# {
#   "ok": true,
#   "redis": true
# }
```

Comprobar (sección 2.6)

Check	Comando	Esperado
API corre	<code>curl http://localhost:8001/healthz</code>	<code>"ok": true,</code> <code>"redis": true</code>
Tests API	<code>pytest soar/approval_api/tests/ -v</code>	<code>>= 8 passed</code>
Endpoint Telegram acepta payload válido	POST con JSON simulado	<code>{"ok": true}</code>
Endpoint Twilio devuelve TwiML válido	<code>curl -X POST 'http://localhost:8001/voice/twiml?incident=test'</code>	XML con <code><Gather></code>

2.7 Two-person rule + conservative-wins policy

Qué estás haciendo

El núcleo del HITL. Cuando un T2 dispara, se notifican N aprobadores (≥ 2). El sistema espera **hasta 2 respuestas concordantes** o cierra la ventana en 60s. La política `conservative-wins` dice: ante conflicto, **prevalece reject**.

Código central completo (`handlers.py`)

```

# soar/approval_api/handlers.py
"""
Lógica HITL: registrar respuestas, evaluar two-person, resolver conflictos.

Estado vive en Redis bajo la clave `incident:{id}`. Cada respuesta se appendea
al array `approvers`. Una vez alcanzado el umbral (2 respuestas concordantes
o timeout), se escribe `final_decision`.

Política conservative-wins (ADR-0006):
- Si ambos approve → execute.
- Si ambos reject → block.
- Si discrepan → block (conservative wins).
- Si solo 1 responde y timeout → block (no quorum).
"""

from __future__ import annotations

import json
import logging
import time
from typing import Literal

import redis.asyncio as redis

from argos_contracts.enums import ApproverStatus, IncidentState
from argos_contracts.incident import (
    ApproverState, FinalDecision, Incident,
)

logger = logging.getLogger(__name__)

QUORUM_NEEDED = 2
DEFAULT_WINDOW_SECONDS = 60

async def _load_incident(r: redis.Redis, incident_id: str) -> Incident:
    raw = await r.get(f"incident:{incident_id}")
    if raw is None:
        raise KeyError(f"incident {incident_id} not in Redis")
    return Incident.model_validate_json(raw)

async def _save_incident(r: redis.Redis, incident: Incident) -> None:
    await r.set(f"incident:{incident.incident_id}", incident.model_dump_json())

async def record_approval_response(
    *, redis_client: redis.Redis,
    incident_id: str,
    approver_id: str,
    decision: Literal["approve", "reject"],
    channel: str,
) -> None:
    incident = await _load_incident(redis_client, incident_id)

```

```

# No-op si ya hay final_decision (idempotencia ante reintentos de webhook)
if incident.final_decision is not None:
    logger.info("incident %s already decided; ignoring late %s from %s",
                incident_id, decision, approver_id)
    return

status = (ApproverStatus.APPROVED if decision == "approve"
          else ApproverStatus.REJECTED)
now = time.time()

# Actualizar o agregar al array de aprobadores
found = False
for ap in incident.approvers:
    if ap.approver_id == approver_id:
        ap.status = status
        ap.responded_at = now
        ap.channel = channel
        found = True
        break
if not found:
    incident.approvers.append(ApproverState(
        approver_id=approver_id,
        status=status,
        channel=channel,
        notified_at=now,
        responded_at=now,
    ))

await _save_incident(redis_client, incident)

def _evaluate(incident: Incident) -> FinalDecision | None:
    """Pure: dada la lista de approvers actual, ¿hay decisión?"""
    approved = sum(a.status == ApproverStatus.APPROVED for a in incident.approvers)
    rejected = sum(a.status == ApproverStatus.REJECTED for a in incident.approvers)
    timeout = sum(a.status == ApproverStatus.TIMEOUT for a in incident.approvers)

    # Caso 1: quorum claro a favor de approve
    if approved >= QUORUM_NEEDED and rejected == 0:
        return FinalDecision(
            outcome="execute",
            policy_applied="two_person_approve",
            execution_status="pending",
            approved_count=approved, rejected_count=rejected, timeout_count=timeout,
        )

    # Caso 2: quorum claro de reject
    if rejected >= QUORUM_NEEDED:
        return FinalDecision(
            outcome="block",
            policy_applied="two_person_reject",
            execution_status="not_required",
            approved_count=approved, rejected_count=rejected, timeout_count=timeout,

```

```

    )

    # Caso 3: conflicto (al menos 1 approve y al menos 1 reject)
    if approved >= 1 and rejected >= 1:
        return FinalDecision(
            outcome="block",
            policy_applied="conservative_wins",
            execution_status="not_required",
            approved_count=approved, rejected_count=rejected, timeout_count=timeout,
        )

    # Caso 4: aún no hay quorum
    return None

async def build_final_decision_if_ready(
    r: redis.Redis, incident_id: str
) -> Incident:
    incident = await _load_incident(r, incident_id)
    if incident.final_decision is not None:
        return incident

    decision = _evaluate(incident)
    if decision is None:
        return incident

    incident.final_decision = decision
    incident.state = (
        IncidentState.RESOLVED if decision.outcome == "block"
        else IncidentState.PENDING_EXECUTION
    )
    await _save_incident(r, incident)
    logger.info("incident %s decided: %s (policy=%s)",
                incident_id, decision.outcome, decision.policy_applied)
    return incident

```

Tests parametrizados (los casos importantes — completos)

```
# soar/approval_api/tests/test_handlers.py
import pytest
from argos_contracts.enums import ApproverStatus
from argos_contracts.incident import ApproverState
from soar.approval_api.handlers import _evaluate

def _ap(decision):
    status = {"approve": ApproverStatus.APPROVED,
              "reject": ApproverStatus.REJECTED,
              "timeout": ApproverStatus.TIMEOUT}[decision]
    return ApproverState(approver_id=decision+"-x", status=status,
                        channel="telegram", notified_at=0, responded_at=0)

@pytest.mark.parametrize("inputs, outcome, policy", [
    # Two approves → execute
    (["approve", "approve"], "execute", "two_person_approve"),
    # Two rejects → block
    (["reject", "reject"], "block", "two_person_reject"),
    # Conflict → conservative-wins block
    (["approve", "reject"], "block", "conservative_wins"),
    (["approve", "reject", "approve"], "block", "conservative_wins"),
    # Only 1 response → no decision yet
    # (handled separately, _evaluate returns None)
])
def test_evaluate_decision_matrix(inputs, outcome, policy, fake_incident):
    inc = fake_incident
    inc.approvers = [_ap(x) for x in inputs]
    decision = _evaluate(inc)
    assert decision is not None
    assert decision.outcome == outcome
    assert decision.policy_applied == policy

def test_single_response_no_decision(fake_incident):
    inc = fake_incident
    inc.approvers = [_ap("approve")]
    assert _evaluate(inc) is None
```

Correr

```
pytest soar/approval_api/tests/test_handlers.py -v
# OUTPUT ESPERADO:
# test_handlers.py::test_evaluate_decision_matrix[inputs0-execute-two_person_approve] PASSED
# test_handlers.py::test_evaluate_decision_matrix[inputs1-block-two_person_reject] PASSED
# test_handlers.py::test_evaluate_decision_matrix[inputs2-block-conservative_wins] PASSED
# test_handlers.py::test_evaluate_decision_matrix[inputs3-block-conservative_wins] PASSED
# test_handlers.py::test_single_response_no_decision PASSED
# ===== 5 passed in 0.04s =====
```

Comprobar (sección 2.7)

Check	Esperado
5 tests de la matriz pasan	sí
Decisión es idempotente (segundo <code>build_final_decision_if_ready</code> no muta nada)	sí
Si llega un response después de <code>final_decision</code> , se ignora con log	sí

2.8 Consolidation window — 60 segundos

Qué estás haciendo

Un job background (asyncio task) que dado un `incident_id` espera 60s y, si no hay quorum, marca a los pendientes como TIMEOUT y llama `build_final_decision_if_ready` una última vez.

Código completo

```

# soar/approval_api/consolidation.py
"""
Ventana de consolidación T2 – 60 segundos.

Se invoca en cuanto se crea un Incident T2. Si al cumplirse el tiempo no hay
final_decision, se marcan los pendientes como TIMEOUT y se evalúa la última
vez. Si la evaluación final no produce decisión (ej. nadie respondió), se
crea una decisión `block` con policy `no_quorum_timeout`.
"""

from __future__ import annotations

import asyncio
import logging
import time

import redis.asyncio as redis

from argos_contracts.enums import ApproverStatus, IncidentState
from argos_contracts.incident import FinalDecision
from soar.approval_api.handlers import (
    _evaluate, _load_incident, _save_incident,
)

logger = logging.getLogger(__name__)

WINDOW_SECONDS = 60

async def consolidation_task(r: redis.Redis, incident_id: str) -> None:
    """Background task. Spawn con `asyncio.create_task(...)`."""
    await asyncio.sleep(WINDOW_SECONDS)

    incident = await _load_incident(r, incident_id)
    if incident.final_decision is not None:
        return # alguien ya cerró antes de los 60s

    # Marcar a los que no respondieron como TIMEOUT
    for ap in incident.approvers:
        if ap.status == ApproverStatus.PENDING:
            ap.status = ApproverStatus.TIMEOUT
            ap.responded_at = time.time()

    decision = _evaluate(incident)
    if decision is None:
        # Nadie respondió o solo timeouts → conservative block
        decision = FinalDecision(
            outcome="block",
            policy_applied="no_quorum_timeout",
            execution_status="not_required",
            approved_count=0, rejected_count=0,
            timeout_count=sum(1 for a in incident.approvers
                             if a.status == ApproverStatus.TIMEOUT),
        )

```



```

incident.final_decision = decision
incident.state = IncidentState.RESOLVED
await _save_incident(r, incident)
logger.info("incident %s: window closed (%s)",
            incident_id, decision.policy_applied)

```

Test (acelerando la ventana con monkeypatch)

```

# soar/approval_api/tests/test_consolidation.py
import asyncio
import pytest

import soar.approval_api.consolidation as consolidation_mod

@pytest.mark.asyncio
async def test_consolidation_no_response_blocks(monkeypatch, fake_redis, fake_incident_t2):
    # Acelerar la ventana de 60s → 0.01s para test
    monkeypatch.setattr(consolidation_mod, "WINDOW_SECONDS", 0.01)

    await fake_redis.set(
        f"incident:{fake_incident_t2.incident_id}",
        fake_incident_t2.model_dump_json(),
    )
    await consolidation_mod.consolidation_task(fake_redis, fake_incident_t2.incident_id)

    raw = await fake_redis.get(f"incident:{fake_incident_t2.incident_id}")
    from argos_contracts.incident import Incident
    final = Incident.model_validate_json(raw)
    assert final.final_decision.outcome == "block"
    assert final.final_decision.policy_applied == "no_quorum_timeout"

```

Comprobar (sección 2.8)

Check	Esperado
<code>pytest soar/approval_api/tests/test_consolidation.py</code>	passed
Si ya hay <code>final_decision</code> antes de 60s, el task no hace nada	sí
Si llegan 0 respuestas, decisión es <code>block / no_quorum_timeout</code>	sí

Checklist Fase 2 — antes de pasar a Fase 3

#	Check	OK
1	<code>pytest soar/</code> corre y pasa todo (esperado ≥ 30 passed)	☐
2	<code>uvicorn soar.approval_api.main:app --port 8001</code> levanta sin errores	☐
3	<code>/healthz</code> devuelve <code>{"ok": true, "redis": true}</code> con Redis local arriba	☐
4	Telegram smoke test envía mensaje real	☐
5	Discord smoke test envía embed real	☐
6	Twilio Voice smoke test produce llamada (al menos una vez)	☐
7	Tier router asigna T0 para técnica <code>T1486</code> y <code>severity=LOW</code>	☐
8	Two-person rule resuelve conflict → <code>block / conservative_wins</code>	☐
9	Consolidation window cierra a <code>no_quorum_timeout</code> cuando nadie responde	☐
10	<code>.env</code> cargado correctamente (sin hardcoded credentials en código)	☐

Si menos de 8/10 está OK, NO avances a Fase 3. Dedica la mitad del día a cerrar lo restante; pinguea en `#argos-help` si te bloqueas.

FASE 3 — Integración real

Goal: tu código habla con el lab Vagrant de verdad (Wazuh manager + Redis + Postgres + las 4 capas). Aquí dejan de ser stubs.

3.1 Consumir eventos reales del lab

Qué estás haciendo

Hasta ahora tu SOAR procesa `NormalizedEvent` fabricados a mano en tests. Ahora va a leerlos de un Redis stream alimentado por las 4 capas (P3 mete Sigma/Canary, P2 mete ML/LLM).

Acuerdo de contrato con P2 y P3 (ya documentado en [docs/contracts/CONTRACTS_SPECIFICATION.md](#))

```
Stream Redis:          events:normalized
Cada XADD:              { "data": <NormalizedEvent JSON> }
Consumer group:        soar-router
Tu consumer name:      soar-1
```

Código central ([soar/decision_engine/consumer.py](#))

```

# soar/decision_engine/consumer.py
"""Consumer del stream Redis events:normalized → emite Incident a Redis."""

from __future__ import annotations

import asyncio, json, logging, os, uuid
from datetime import datetime, timezone

import redis.asyncio as redis

from argos_contracts.enums import IncidentState
from argos_contracts.incident import HostInfo, Incident, NormalizedEvent
from soar.decision_engine.tier_router import route
from soar.notifications.service import NotificationService

logger = logging.getLogger(__name__)

STREAM      = "events:normalized"
GROUP       = "soar-router"
CONSUMER    = os.environ.get("SOAR_CONSUMER_NAME", "soar-1")
INCIDENT_TTL_SECONDS = 6 * 3600 # 6 horas

async def ensure_group(r: redis.Redis) -> None:
    try:
        await r.xgroup_create(STREAM, GROUP, id="0", mkstream=True)
    except redis.ResponseError as e:
        if "BUSYGROUP" not in str(e):
            raise

async def _process_event(
    r: redis.Redis, notif: NotificationService, raw: dict
) -> None:
    event = NormalizedEvent.model_validate_json(raw["data"])
    tier = route(event)
    incident = Incident(
        incident_id=f"inc-{uuid.uuid4().hex[:12]}",
        tier=tier,
        state=IncidentState.NEW,
        severity=event.severity,
        host=HostInfo(hostname=event.host, ip="", os=""),
        mitre_technique=event.mitre_technique,
        num_layers_fired=event.num_layers_fired,
        confidence_score=event.confidence_score,
        created_at=datetime.now(timezone.utc),
        requires_approval=(tier == tier.T2),
        approvers=[], final_decision=None, consolidation_window=None,
    )
    await r.setex(
        f"incident:{incident.incident_id}",
        INCIDENT_TTL_SECONDS,
        incident.model_dump_json(),
    )

```

```

        results = notif.dispatch_for_tier(incident)
        logger.info("incident %s → tier=%s notif=%s",
                    incident.incident_id, tier.value,
                    [(r.channel.value, r.success) for r in results])

async def run_consumer(notif: NotificationService) -> None:
    r = redis.from_url(os.environ["REDIS_URL"], decode_responses=True)
    await ensure_group(r)
    logger.info("consumer %s reading group %s on stream %s", CONSUMER, GROUP, STREAM)
    while True:
        resp = await r.xreadgroup(
            GROUP, CONSUMER, {STREAM: ">"}, count=10, block=5000
        )
        for _stream, entries in resp or []:
            for entry_id, fields in entries:
                try:
                    await _process_event(r, notif, fields)
                    await r.xack(STREAM, GROUP, entry_id)
                except Exception:  # noqa: BLE001
                    logger.exception("failed to process %s; entry retained", entry_id)
                    # No-ack → reintento próximo xreadgroup tras min-idle.

```

Probar end-to-end con un evento inyectado manualmente

```

# Terminal A: levanta el consumer
python -m soar.decision_engine.consumer
# OUTPUT ESPERADO:
# INFO:soar.decision_engine.consumer:consumer soar-1 reading group soar-router on stream events:normalized

# Terminal B: inyecta un evento simulado
redis-cli XADD events:normalized '*' data '{"event_id":"evt-001","severity":"CRITICAL","mitre_technique":"WIN-VICTIM-01"}'
# OUTPUT ESPERADO:
# "1716580800000-0" (timestamp-secuencia)

# En terminal A debes ver:
# INFO:soar.decision_engine.consumer:incident inc-abc123def456 → tier=T0 notif=[('telegram', True), ('win-victim-01', True)]

# Verificar incident persistido en Redis
redis-cli KEYS 'incident:*'
# OUTPUT ESPERADO:
# 1) "incident:inc-abc123def456"

redis-cli GET incident:inc-abc123def456 | jq .tier,.host.hostname,.mitre_technique
# OUTPUT ESPERADO:
# "T0"
# "WIN-VICTIM-01"
# "T1486"

```

Comprobar (sección 3.1)

Check	Esperado
Consumer arranca sin errores	sí
Evento inyectado → genera <code>incident:*</code> en Redis	sí
Tier asignado coincide con la matriz de <code>tier_router</code>	sí
Notificaciones llegan a Telegram + Discord (canales aparecen <code>success=True</code>)	sí

3.2 Audit log en PostgreSQL

Qué estás haciendo

Cada `Incident` y cada respuesta de aprobador debe persistirse en PostgreSQL (P4 levantó la DB con `pgaudit` activo en su Fase 1). Razón: trazabilidad forense post-mortem.

Schema (acordado con P4 — ya está en `lab/postgres/init.sql`)

```
CREATE TABLE IF NOT EXISTS audit_incidents (  
    incident_id    text PRIMARY KEY,  
    tier           text NOT NULL,  
    severity       text NOT NULL,  
    host          text NOT NULL,  
    technique      text NOT NULL,  
    created_at     timestampz NOT NULL,  
    final_outcome  text,  
    final_policy   text,  
    final_at       timestampz,  
    payload        jsonb NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS audit_responses (  
    id             bigserial PRIMARY KEY,  
    incident_id    text REFERENCES audit_incidents(incident_id),  
    approver_id    text NOT NULL,  
    channel        text NOT NULL,  
    decision       text NOT NULL,  
    received_at    timestampz NOT NULL  
);
```

Persistencia (boilerplate, skeleton + referencia)

```
# soar/audit/postgres_sink.py
"""
Sink de auditoría hacia PostgreSQL.

Se llama dos veces:
- al crear el incidente (insert)
- al cerrar la final_decision (update)
- cada vez que llega una respuesta de aprobador (insert en audit_responses)

Usar asyncpg (async, pooled). El pool se crea en startup de la app.

Para detalles del esquema y migrations, ver `lab/postgres/init.sql` (owner: P4).
La función a llamar desde `consumer.py` es:

    await persist_new_incident(pool, incident)
    await persist_final_decision(pool, incident)
    await persist_response(pool, incident_id, approver_id, channel, decision, ts)

Patrón estándar: parametrized queries, JSONB para el payload completo,
upsert con ON CONFLICT en `incident_id`. Ver test_postgres_sink.py para fixtures.
"""

# (Skeleton – completa siguiendo el patrón conocido. ~80 LOC totales.)
```

Comprobar (sección 3.2)

```
# Con el lab arriba (Vagrant levantado por P4):
psql postgresql://argos:argos@localhost:5432/argos_audit -c \
  "SELECT incident_id, tier, technique, final_outcome FROM audit_incidents ORDER BY created_at DESC"
# OUTPUT ESPERADO (después de correr una simulación):
#   incident_id      | tier | technique | final_outcome
# -----+-----+-----+-----
# inc-abc123def456   | T0   | T1486     | execute
# ...
```

Check	Esperado
Tabla <code>audit_incidents</code> se llena tras un evento	sí
Tabla <code>audit_responses</code> se llena tras una aprobación	sí
<code>pgaudit</code> registra los INSERT (verificable con <code>SELECT * FROM pg_stat_activity</code>)	sí

3.3 Hook a LLM Triage (Layer 4)

Qué estás haciendo

Cuando el `tier_router` asigna T2 (no T0, no T3), invocas el LLM Triage que P2 expone como módulo `ml.llm_triage.classify(event)` y devuelve un campo extra `llm_verdict` que enriquece el `Incident`.

Patrón de integración

```
# soar/decision_engine/consumer.py – modificación en _process_event:

# ... después de calcular tier:
if tier == Tier.T2:
    from ml.llm_triage import classify as llm_classify
    try:
        verdict = await llm_classify(event) # P2 expone esto
        # verdict = LLMVerdict(label=..., reasoning=..., confidence=...)
    except Exception: # noqa: BLE001
        logger.exception("llm classify failed; continuing without enrichment")
        verdict = None
    incident.llm_verdict = verdict
```

Importante: si el LLM falla (rate-limit, timeout, OpenAI down), el incidente sigue su curso sin enriquecimiento. **Nunca bloquee** el flujo HITL por el LLM.

Comprobar (sección 3.3)

Check	Esperado
Para evento T2, <code>incident.llm_verdict</code> está poblado en Redis	sí
Si <code>LLM_BACKEND=llama_local</code> y Ollama no corre, el flujo no se cae (verdict=None)	sí

3.4 End-to-end UC-01 (ransomware T0)

Comando del demo

```
# Terminal 1 – consumer
python -m soar.decision_engine.consumer

# Terminal 2 – approval API
uvicorn soar.approval_api.main:app --port 8001

# Terminal 3 – lanzar ataque
python attack-simulation/ransomware_simulator/lockbit_like.py \
    --variant uc01 --target windows-victim

# OUTPUT ESPERADO (en terminal 1):
# INFO:soar.decision_engine.consumer:incident inc-XXX → tier=T0 notif=[('telegram', True), ('discord', True)]
#
# OUTPUT ESPERADO (en Streamlit Console http://localhost:8501):
# Incident card con tier=T0, técnica=T1486, host=WIN-VICTIM-01,
# y al cabo de pocos segundos: state=PENDING_EXECUTION → execution_status=success
# (acción: aislar host vía Wazuh active response).
```

Comprobar (sección 3.4)

Check	Esperado
Tiempo desde lanzar ataque → notificación Telegram ≤ 8s	sí
Tiempo desde lanzar ataque → host aislado (red bloqueada) ≤ 15s	sí
<code>incident.final_decision.policy_applied == "auto_t0"</code> (sin pasar por HITL)	sí
Postgres tiene fila correspondiente en <code>audit_incidents</code>	sí

3.5 End-to-end UC-04 (two-person rule)

Comando

```
# Terminales 1 y 2 como arriba.
# Terminal 3:
python attack-simulation/ransomware_simulator/postgres_attack.py \
    --target linux-victim
```

Lo que debería pasar (timeline)

```
t=0s    Ataque lanzado. Wazuh dispara regla Sigma → evento normalizado.
t=2s    SOAR consumer crea Incident T2 → notifica Telegram + Discord.
        Los 4 integrantes reciben mensaje con botones [Approve] [Reject].
t=10s   P1 toca [Approve] → Approval API registra → 1 approve.
t=18s   P2 toca [Approve] → 2 approves → final_decision=execute → acción ejecuta.
        Streamlit muestra banner verde: "2 approve · 0 reject · execute".
```

Variante con conflict

```
t=10s   P1 toca [Approve]
t=18s   P2 toca [Reject]
        → conservative-wins → final_decision=block / policy=conservative_wins
        → Streamlit muestra banner gris: "1 approve · 1 reject · block".
```

Variante con timeout

```
t=10s   P1 toca [Approve]
t=60s   ventana cierra → P2/P3/P4 marcados TIMEOUT
        → no quorum → final_decision=block / policy=no_quorum_timeout
        → en t=60s también dispara Twilio Voice a los que no respondieron
          (si la ventana es ≥ 60s, la llamada sale a t=55s).
```

Comprobar (sección 3.5)

Check	Esperado
Los 4 integrantes reciben Telegram	sí
Botones aparecen y son clickeables	sí
2 approves → execute en $\leq 20s$	sí
Conflict caso → block / conservative_wins	sí
Timeout caso → block / no_quorum_timeout	sí
Twilio dispara a t=55-60s para no-respondientes	sí

Checklist Fase 3 — antes de pasar a Fase 4

#	Check	OK
1	Consumer SOAR consume del stream real y emite incidents a Redis	☐
2	UC-01 corre end-to-end (T0 auto) en < 15s p95	☐
3	UC-04 corre end-to-end con 2 approves → execute	☐
4	UC-04 conflict → block / conservative_wins	☐
5	UC-04 timeout → block / no_quorum_timeout	☐
6	Audit log en PostgreSQL refleja cada incident + cada response	☐
7	LLM hook degrada graciosamente si OpenAI cae	☐
8	<code>pytest -q</code> global pasa (esperado 80+ tests)	☐

FASE 4 — Rehearsal y polish

Goal: simular el demo completo dos veces. La segunda vez con todos los integrantes presentes en llamada Discord. Documentar cada fallo y resolverlo antes del demo real.

4.1 Rehearsal #1 — solo P1 (silencioso)

Procedimiento

```
# 1. Vagrant arriba, todos los servicios up
make demo-up

# 2. Cronómetro arriba (usa `time` en bash o un timer real)
time {
  python attack-simulation/ransomware_simulator/lockbit_like.py --variant uc01 --target windows-victim
  sleep 20 # esperar ejecución
  python attack-simulation/ransomware_simulator/canary_path.py --target linux-victim
  sleep 15
  python attack-simulation/ransomware_simulator/postgres_attack.py --target linux-victim
  # esperar two-person en otro terminal
}

# Métricas a registrar:
# - tiempo total desde primer comando hasta último incident.state=RESOLVED
# - número de notificaciones que fallaron (revisar logs del consumer)
# - cualquier excepción no manejada (grep ERROR en logs)
```

Reportar resultado en standup

UC	Tiempo p50	Tiempo p95	Fallos
UC-01	7s	12s	0
UC-02	5s	8s	0
UC-04 (2-approve)	22s	35s	0

Cualquier fallo → ticket → resolver antes de rehearsal #2.

4.2 Rehearsal #2 — con los 4 integrantes (call Discord)

Preparación (T-30 min)

1. Crear evento Discord call: "ARGOS Rehearsal Final" a hora X.
2. Cada integrante con su celular cargado, Telegram + Discord abiertos.
3. P4 confirma que su lab está arriba y que tu lab espejo también.
4. Tú compartes pantalla con Streamlit Console.

Script del ensayo (12-13 min)

Tiempo	Acción	Quién
0:00	"Comenzamos UC-01"	P1 (tú) narras
0:05	<code>python ...lockbit_like.py ...</code>	P4 ejecuta
0:30	Mostrar Streamlit con incident T0	P1 explica capas firing
2:00	"UC-02 ahora"	P1
2:05	<code>python ...canary_path.py ...</code>	P4
4:00	"UC-04 — todos atentos al celular"	P1
4:05	<code>python ...postgres_attack.py ...</code>	P4
4:10	Telegram llega — P1 aprueba	P1
4:18	P2 aprueba	P2
4:30	Mostrar banner "execute"	P1 explica policy
6:00	Repetir UC-04 — esta vez P1 approve, P2 reject (conflict)	mismo guion
8:00	Repetir UC-04 — nadie responde (timeout + Twilio)	celular suena
10:00	Q&A simulado	todos

Comprobar (sección 4.2)

Check	Esperado
Ningún integrante reporta no haber recibido Telegram	sí
Streamlit Console refresca en $\leq 2s$ tras cada acción	sí
Llamada Twilio entra a celular acordado	sí
Audit log final tiene los 3 escenarios UC-04 distinguibles	sí
Total ≤ 13 minutos	sí

4.3 Edge cases y contingencias

Tabla de fallos previstos y respuesta

Fallo	Probabilidad	Respuesta inmediata
WiFi del salón bloquea api.openai.com	Alta	Cambiar <code>LLM_BACKEND=llama_local</code> y reiniciar el LLM Triage (P2)
Telegram API rate limit (>30 msgs/s)	Baja	No es escenario probable — solo demos manuales. Si pasa, Discord aún funciona.
Twilio trial sin saldo	Media	Verificar saldo T-24h. Si <\$3, top-up con \$5 más.
Wazuh manager no arranca	Media	Tu lab espejo. Switch con <code>export WAZUH_HOST=p1-espejo.local</code> y reiniciar consumer.
Postgres lleno (>90%)	Baja	<code>make audit-vacuum</code> (script que limpia incidents > 24h en demo)
Crash del SOAR consumer mid-demo	Media	Script <code>make soar-restart</code> (systemd-like, reinicia en < 5s sin perder el offset del stream gracias al group)
Streamlit no actualiza	Media	Refresh manual F5 — los datos están en Redis, la UI sólo lee
Llamada Twilio se va a buzón de voz	Alta	Por diseño: si nadie levanta, decisión cae a <code>no_quorum_timeout</code> automáticamente

Plan de fallback nuclear

Si **3 fallos consecutivos** durante la primera parte del demo:

1. Detener intentos en vivo.
2. Mostrar video pre-grabado de rehearsal #2 (graba con OBS antes).
3. P1 narra sobre el video.
4. Q&A se hace al final con sistema arriba o no.

Tener el video listo es no-negociable. Grábalo en rehearsal #2 con OBS.

4.4 Checklist pre-demo (T-2 horas)

#	Check	OK
1	Lab primario (P4) — <code>vagrant status</code> todos UP	☐
2	Lab espejo (P1) — <code>vagrant status</code> todos UP	☐
3	Telegram bot envía mensaje de prueba a los 4 celulares	☐
4	Discord webhook envía mensaje al canal	☐
5	Twilio: credit > \$3 USD	☐
6	OpenAI: usage budget no excedido	☐
7	Postgres responde a <code>psql -c "\dt"</code>	☐
8	OpenSearch dashboard carga en http://localhost:5601	☐
9	Streamlit Console abre en http://localhost:8501	☐
10	Video respaldo grabado y accesible offline	☐
11	Pendrive USB con repo clonado (para caso laptop totalmente muerta)	☐
12	Cargadores de las 2 laptops (P1 + P4)	☐

Checklist Fase 4 — listo para el demo

#	Check	OK
1	Rehearsal #1 cerrado sin issues abiertos	☐
2	Rehearsal #2 cerrado con los 4 integrantes presentes	☐
3	Video respaldo grabado	☐
4	Checklist 4.4 corrido T-2 horas	☐
5	Lessons learned actualizado en <code>docs/LESSONS_LEARNED.md</code>	☐

Apéndice A — Troubleshooting común

A.1 Prereqs no instalados / versión equivocada

OS	Comando
Ubuntu/ Debian	<code>sudo add-apt-repository ppa:deadsnakes/ppa && sudo apt update && sudo apt install python3.11 python3.11-venv python3.11-dev redis-tools jq</code>
macOS (brew)	<code>brew install python@3.11 redis jq</code>
Windows	Usa WSL2 Ubuntu — mismas instrucciones que Ubuntu

A.2 `argos_contracts` no importa después de `pip install -e`

Síntoma: `ModuleNotFoundError: No module named 'argos_contracts'`. Causa común: instalaste sin venv activo o desde la carpeta equivocada.

```
which python    # debe terminar en .venv/bin/python
cd ~/code/argos
pip install -e ./argos_contracts --force-reinstall
python -c "import argos_contracts; print(argos_contracts.__file__)"
# Debe apuntar a tu repo, no a site-packages global.
```

A.3 Telegram devuelve "chat not found"

Causa: nunca enviaste un mensaje al bot desde tu cuenta, entonces `getUpdates` no devuelve tu chat_id real.

Fix: abre el bot en Telegram (link `t.me/<usuario_bot>`), envía `/start`, luego reconsulta `getUpdates`.

A.4 Discord webhook 401

Causa: webhook URL mal copiada (te falta el token al final) o el canal fue eliminado.

Fix: recrear el webhook (Settings → Integrations → Webhooks → New).

A.5 Twilio error 21219 — `Number not verified`

Causa: Twilio requiere que `T0` esté en la lista de "verified numbers".

Fix: <https://console.twilio.com/us1/develop/phone-numbers/manage/verified>

A.6 Redis **CONNECTION REFUSED**

Causa: Redis no está corriendo en el host esperado.

```
# ¿corre algún Redis local?
ss -tlnp | grep 6379
# Si no aparece nada:
docker run -d -p 6379:6379 --name argos-redis redis:7
redis-cli ping
# OUTPUT ESPERADO: PONG
```

A.7 **BUSYGROUP Consumer Group name already exists**

No es error real: significa que el grupo ya estaba creado de una corrida previa. El código ignora este caso. Si sigue molestando:

```
redis-cli XGROUP DESTROY events:normalized soar-router
redis-cli XGROUP CREATE events:normalized soar-router 0 MKSTREAM
```

A.8 Approval API **RuntimeError: Event loop is closed**

Causa: alguna fixture de test cerró el loop antes que el cliente Redis terminara.

Fix: en `conftest.py`, usa `asyncio_mode = "auto"` en `pyproject.toml` y `pytest-asyncio==0.23.x`. Verificar `pip show pytest-asyncio`.

A.9 Tests **xfail** que pasan

Si ves **XPASSED** en algún test marcado como `@pytest.mark.xfail`, **bórrale** el mark. Significa que el bug fue resuelto y el test ya pasa.

A.10 Latencias notif > 2s (lento sospechoso)

```
# Mide round-trip a la API externa:
time curl -s https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/getMe > /dev/null
# Esperado < 800ms en una conexión decente.
# Si >2s: DNS lento, o tu hotspot de celular tiene mala latencia → cambiar red.
```

Apéndice B — Comandos de emergencia (durante demo)

```
# Reiniciar todo el plano de control SOAR sin tocar el lab
make soar-restart
# (alias de: pkill -f 'uvicorn soar.approval_api' ; pkill -f 'soar.decision_engine.consumer' ;
#          sleep 1 ; nohup ... > logs/soar.log 2>&1 & )

# Switch a Llama local si OpenAI cae
export LLM_BACKEND=llama_local && make llm-restart

# Switch al lab espejo
export WAZUH_HOST=p1-espejo.local && make soar-restart

# Limpiar incidentes del demo previo (para empezar limpio)
redis-cli --scan --pattern 'incident:*' | xargs -r redis-cli DEL

# Resetear consumer group offset (re-procesa últimos eventos)
redis-cli XGROUP SETID events:normalized soar-router 0

# Forzar block manual de un incident colgado
redis-cli SET 'incident:inc-XXX:override' 'force_block'
# (handler en `consumer.py` chequea este flag al cargar incident)
```

Apéndice C — Referencias cruzadas

Cuando estés en...	Lee
<code>tier_router.py</code>	SAD §6.2 (Decision Engine), ADR-0002 (tier matrix)
<code>notifications/</code>	ADR-0007 v2 (multi-channel + escalación), SAD §8.3
<code>approval_api/</code>	ADR-0006 (split-brain + conservative-wins), USE_CASES UC-04
<code>consumer.py</code>	<code>docs/contracts/CONTRACTS_SPECIFICATION.md</code> §4 (Redis streams)
<code>audit/</code>	THREAT_MODEL §5 (trazabilidad), SAD §11 (audit)
Contingencias del demo	<code>docs/team/sprint-week-1-overview.md</code> §1 + THREAT_MODEL §7

Cuando edites un ADR o el SAD durante la semana: NO. Esta semana está congelado (regla de oro #1 del overview). Cualquier cambio arquitectónico abre ADR nuevo después del demo.

Change log de este manual

Versión	Fecha	Cambio	Autor
2.0	2026-05-24	Reorganización completa: estructura day-by-day → estructura por feature (Fase 1-4). Agregados expected outputs literales después de cada comando, checklists al final de cada sección, troubleshooting con 10 entradas, comandos de emergencia para demo.	P1